

Universidad Nacional de San Agustín
Escuela Profesional de Ingeniería de Sistemas

Diseño y Arquitectura y de Software

M. Sc. Percy Huertas Niquén

Proyectos del curso

- Formación de proyectos grupales (promedio: 7 – 12 estudiantes)
- Cada grupo debe entregar los siguientes artefactos:
  - Definir el líder del proyecto
  - Elaborar el plan de trabajo (firmada por el líder)
  - Elaborar las actas semanales/mensuales (firmadas por los integrantes)
  - Elaborar el catálogo de requisitos
  - Elaborar la gestión de riesgo de los requisitos
  - Elaborar la arquitectura del sistema
- Cumplir con el objetivo previsto en las etapas

Objetivos por etapas

- Primera fase (evaluación continua 1 y examen 1)
  - Actas de trabajo firmadas
  - Catálogo de requisitos
  - Arquitectura genérica
  - Diagrama de comportamiento
- Segunda fase (evaluación continua 2 examen 2)
  - Actas de trabajo firmadas
  - Vista lógica
  - Vista de procesos
  - Vista física
- Tercera fase (evaluación continua 3 y examen 3)
  - Actas de trabajo firmadas
  - Arquitectura centrada en interfaces de usuario
  - Arquitectura centrada en datos
  - Arquitectura centrada en el flujo de datos

Explica, desarrolla y comparte las especificaciones técnicas y tecnológicas de la construcción de productos de software basados en las necesidades del cliente y el usuario final.
Define, representa, conoce, utiliza y comparte técnicas, habilidades, metodologías y herramientas que permiten la construcción de productos de software durante el ciclo de vida en la construcción de productos software.

Define, representa, conoce, utiliza y comparte técnicas, habilidades, metodologías y herramientas que permiten la construcción de productos de software durante el ciclo de vida en la construcción de productos software.
Analiza, diseña, implementa, valida, prueba e implanta productos de software, que satisfaga necesidades del cliente y el usuario final y elaborados por medio de un proceso de desarrollo de software .

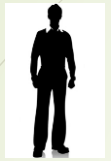
Analiza, diseña, implementa, valida, prueba e implanta productos de software, que satisfaga necesidades del cliente y el usuario final y elaborados por medio de un proceso de desarrollo de software

Motivación

- Construcción de una casa ¿Cómo se hace?

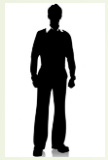


Construcción de una casa



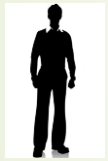
Dueño

- Terreno
- Dinero
- Condicionantes de diseño
- Profesionales



Arquitecto

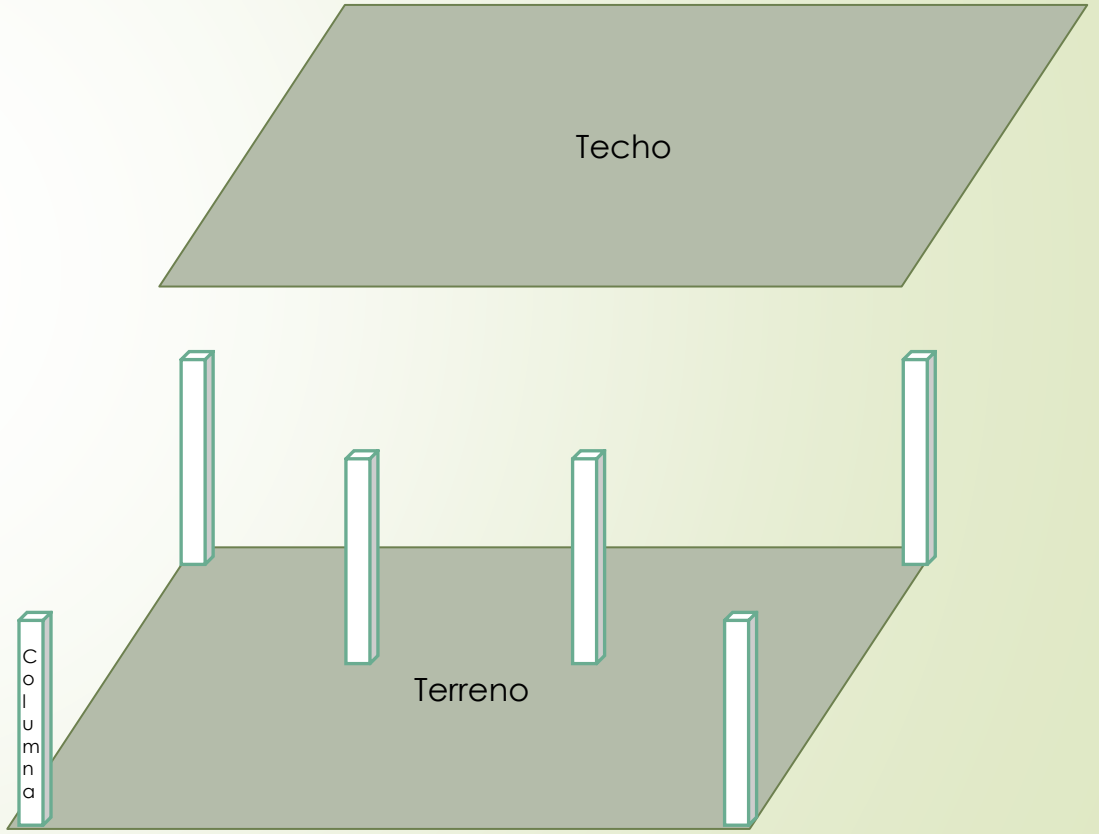
- Distribución
- Diseño
- Propuestas visuales



Ing. Civil

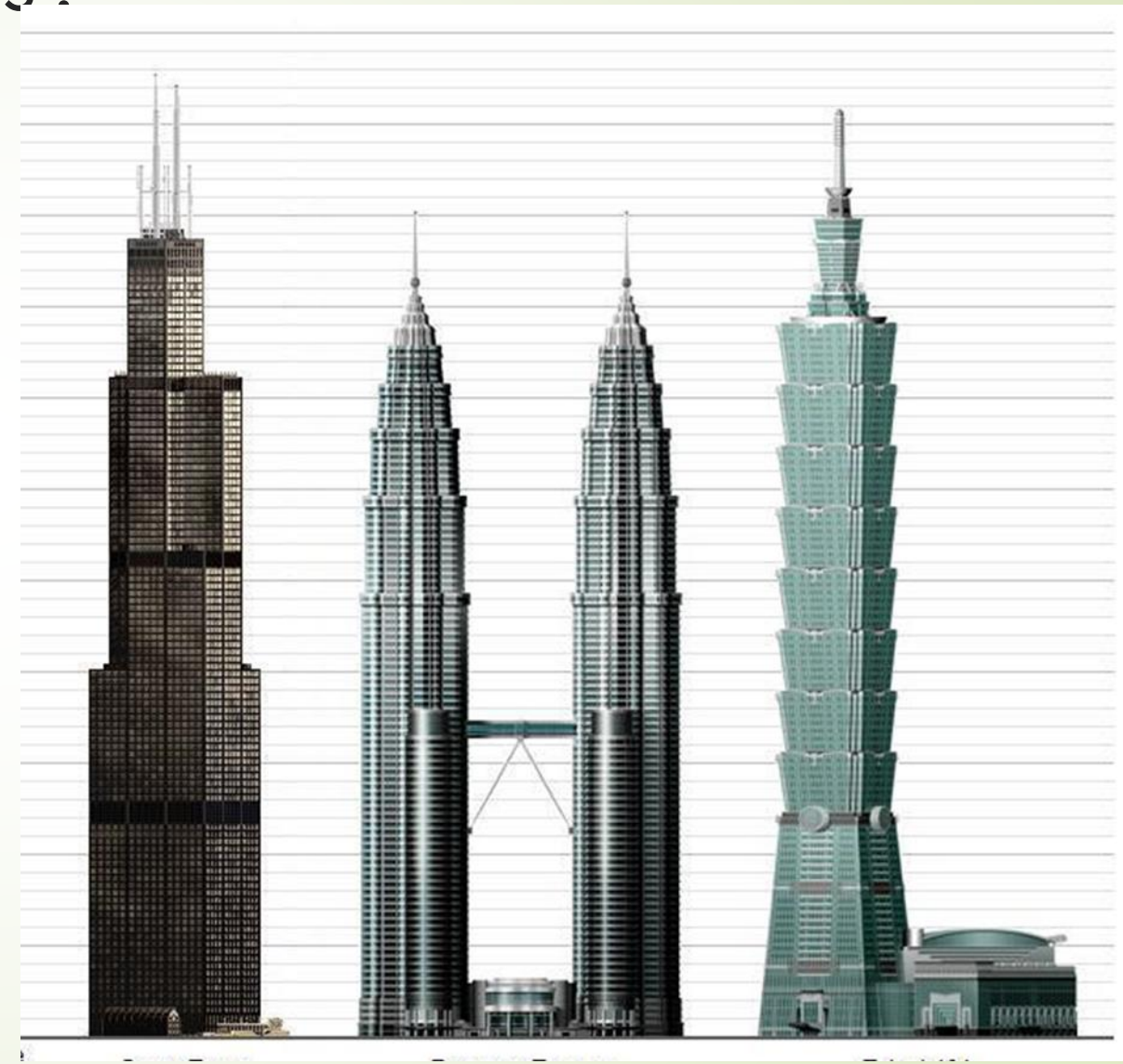
- Diseño de estructuras
- Ubicación de estructuras
- Propuestas del recurso

¿Qué pasa si movemos las columnas?



¿Cómo hacerlos?

- Sears EEUU mide 527 metros
- Petrona Malasia 452 metros
- Taipei 101 China 508 metros



¿Qué tan fácil es modificar?

- Me gustaría que el ascensor quedara del otro lado
- Estaría bárbaro que el puente estuviera 23 pisos más arriba, la vista sería mejor



¿Qué tan fácil es modificarla?

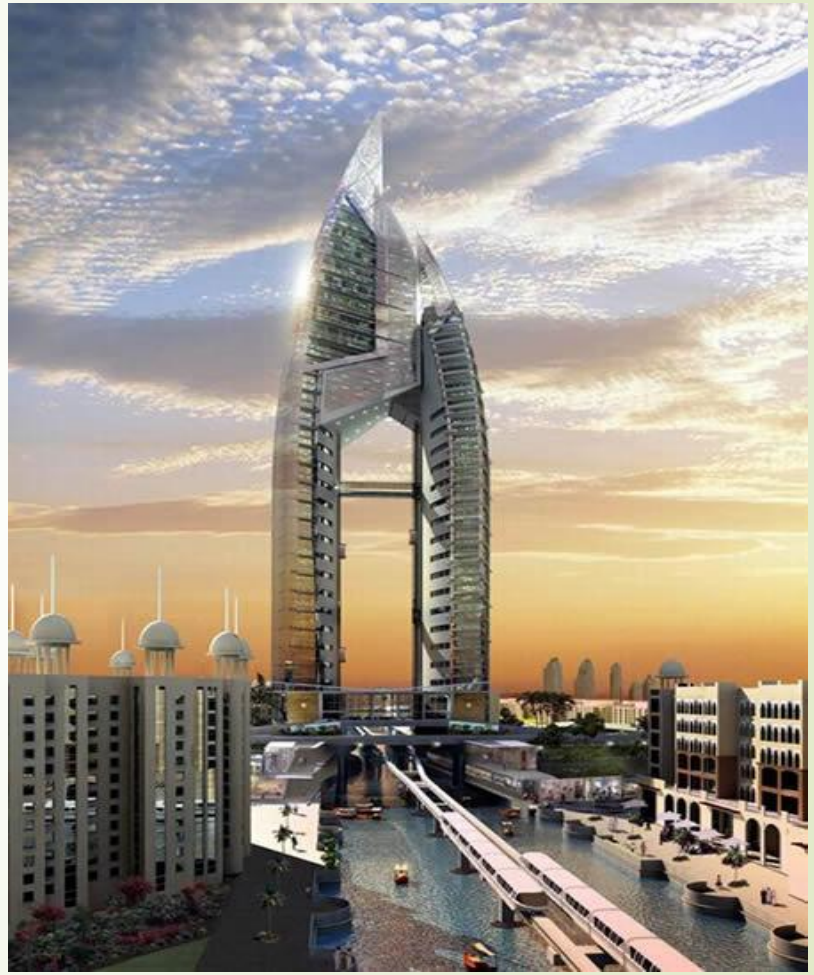
- Burj Dubai, otros metros más arriba, Emiratos Árabes





Más complicado



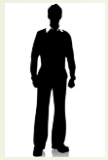


Construcción de un software



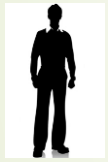
Cliente

- Necesidades



Arquitecto

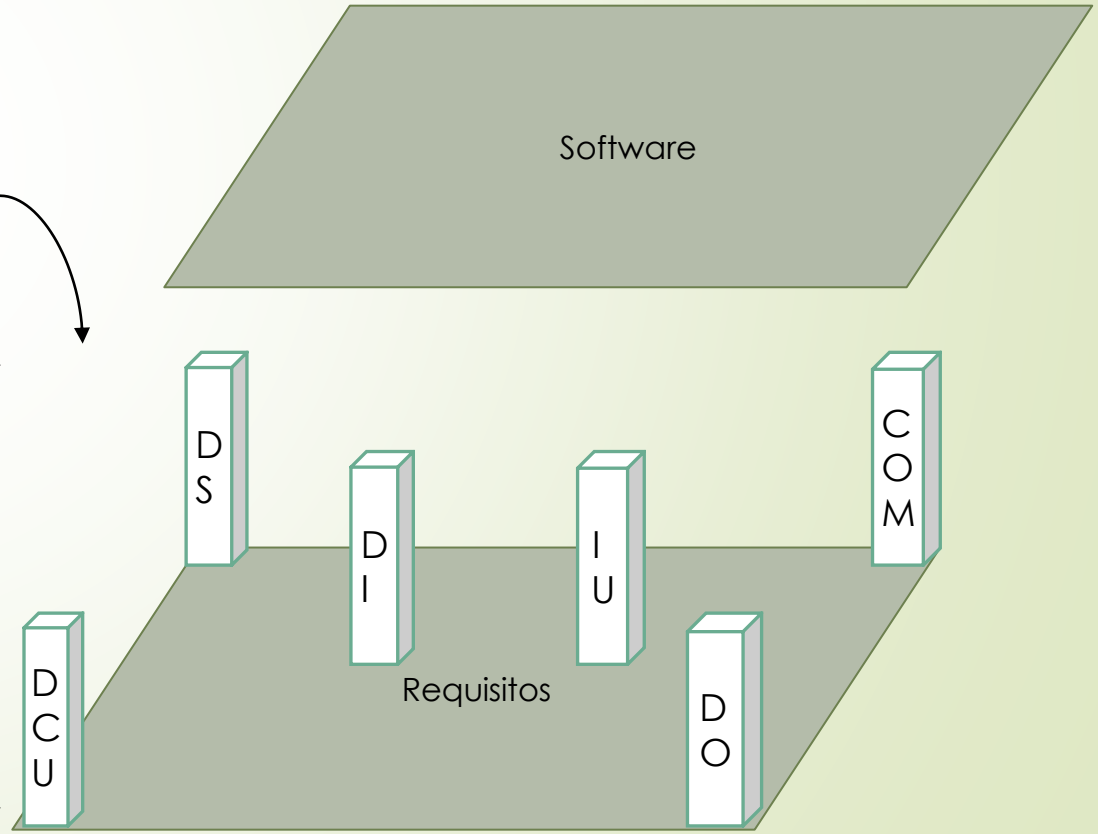
- Propone estructuras



Codificador

- Codifica las estructuras

ARQUITECTURA DE SOFTWARE



Proyectos del curso

- Formación de proyectos grupales (promedio: 7 – 10 estudiantes)
- Cada grupo debe entregar los siguientes artefactos:
  - Definir el líder del proyecto
  - Elaborar el plan de trabajo (firmada por el líder)
  - Elaborar las actas semanales/mensuales (firmadas por los integrantes)
  - Elaborar el catálogo de requisitos
  - Elaborar la gestión de riesgo de los requisitos
  - Elaborar la arquitectura del sistema
- Cumplir con el objetivo previsto en las etapas

Objetivos por etapas

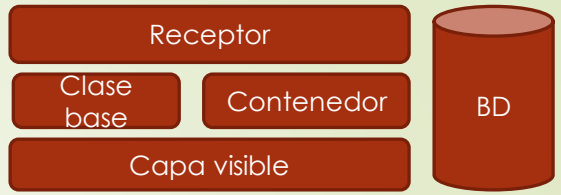
- Primera fase (evaluación continua 1 y examen 1)
  - Actas de trabajo firmadas
  - Catálogo de requisitos (versión revisada del semestre anterior)
  - Arquitectura genérica
  - Diagrama de comportamiento
- Segunda fase (evaluación continua 2 examen 2)
  - Actas de trabajo firmadas
  - Vista lógica
  - Vista de procesos
  - Vista física
- Tercera fase (evaluación continua 3 y examen 3)
  - Actas de trabajo firmadas
  - Arquitectura centrada en interfaces de usuario
  - Arquitectura centrada en datos
  - Arquitectura centrada en el flujo de datos

Conocimientos previos

- Análisis de sistemas
- Diseño de sistemas
- Lenguaje de modelado
- Ingeniería de requerimientos
- Introducción a la Ingeniería de Software

Índice

1. Introducción a la ingeniería de requerimientos
2. Introducción a la arquitectura de software
3. Diseño de arquitectura de software
4. Modelos de arquitectura de software
5. Paradigma orientado a objetos
6. Arquitectura orientada al flujo de datos
7. Arquitectura de software centrado en el dato
8. Arquitectura jerárquica
9. Arquitectura de software para comunicación asíncrona
10. Arquitectura de software orientada a la interacción
11. Arquitectura distribuida
12. Arquitectura de software orientada a componentes
13. Arquitecturas heterogéneas
14. Arquitectura de interfaces gráficas de usuario
15. Arquitectura en la nube



Parte 1: Introducción a la ingeniería de requerimientos

M.Sc. Percy Huertas Niquén

Elicitación de requerimientos

1. Educción de requerimientos
2. Ilación de requerimientos
3. Especificación de requerimientos

Ejemplo de educción

EDU 00001	GESTIÓN DE SISTEMA	
Versión	2.1	
Tipo	Primario	
Objetivo	Controlar el acceso al programa principal, como también a un proyecto ya creado.	
Fecha	19/11/2015	
Fuente	**Nombre**	Percy Huertas
	Cargo	Líder del proyecto
	Tipo	Externo
Especialista	**Nombre**	Vargas Huamán José Miguel
	Especialidad	Ing. Requisitos
	Experiencia	Media
	Cargo	Analista
Descripción	El cliente solicita el acceso al programa principal, como también a un proyecto ya creado.	
Observaciones	-	

Ejemplo de elicitación

ELI – 00001	GESTIÓN DE PROGRAMA PRINCIPAL
EDUCCIÓN Nº	EDU – 00001
VERSION	1.0
FECHA	19/11/2015
AUTOR(ES)	Anco Huamán Erika, Incalla Nina Christian
FUENTES	Stakeholder Percy Huertas
DEPENDENCIAS	Departamento de Diseño
DESCRIPCIÓN	El acceso a programa principal será libre, cualquier persona que tenga acceso al ordenador donde está instalado podrá manipularlo.
PRECONDICIÓN	EL programa debe estar correctamente instalado en el ordenador.
SECUENCIA NORMAL	1 El usuario podrá iniciar la aplicación REMAN.
	2 El usuario podrá cerrar la aplicación REMAN.
POSTCONDICIÓN	El usuario está ejecutando el software REMAN
EXCEPCIONES	-
COMENTARIOS	No se precisa si se podrán compartir proyectos entre diferentes maquinas, considerar trabajos posteriores control de concurrencia.

Ejemplo de especificación

ERS – 00001	INGRESAR A PROGRAMA PRINCIPAL
ELICITACIÓN Nº	ELI – 00001
VERSION	1.0
FECHA	23/11/2015
AUTOR(ES)	Vargas Huamán José Miguel
FUENTES	Stakeholder Percy Huertas
DEPENDENCIAS	Departamento de Diseño
DESCRIPCIÓN	El usuario podrá acceder al programa principal si este se encuentra correctamente instalado.
PRECONDICIÓN	El sistema debe estar correctamente instalado.
POSTCONDICIÓN	El sistema muestra la interfaz principal siendo la opción a primera vista la selección de un proyecto para continuar con la edición, en caso sea el primer uso el sistema mostrará alguna imagen a modo de bienvenida y solo le quedará al usuario registrar el primer proyecto, como también las organizaciones y actores.
EXCEPCIONES	El sistema se encuentra mal instalado en el computador, o con archivos faltantes, lo que conlleva a no poder iniciar el sistema.
COMENTARIOS	Las consideraciones de cómo se muestre la lista de proyectos es libre decisión de Diseño. Solo tener en cuenta que no es necesario abrir algún proyecto, para ingresar una organización o actores, estos datos son generales, al momento de crear un proyecto se hace referencia a estos datos.



Parte 2: Introducción al diseño de arquitectura de software

M.Sc. Percy Huertas Niquén



Introducción a la arquitectura de software

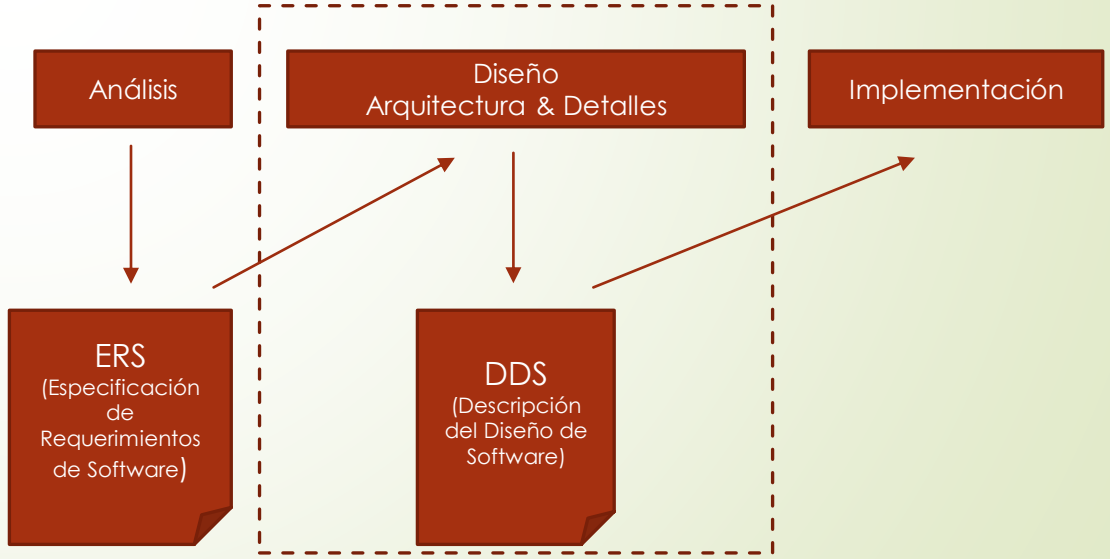
M.Sc. Percy Huertas Niquén

Introducción

- El objetivo de todo DISEÑO DE SOFTWARE es construir un MODELO que refleje todas las necesidades del cliente permitiendo implementaciones reales.
- Todo SOFTWARE debe contener una ARQUITECTURA que guie su construcción.
- El DISEÑO ARQUITECTONICO define la relación entre los mayores elementos estructurales del software. Los estilos y patrones son usados para representar los requerimientos.
- El diseño de la ARQUITECTURA DE SOFTWARE es una derivación de los REQUERIMIENTOS y del ANALISIS DEL SISTEMA.
- Responsable de la tarea: INGENIERO DE REQUERIMIENTOS.

Introducción

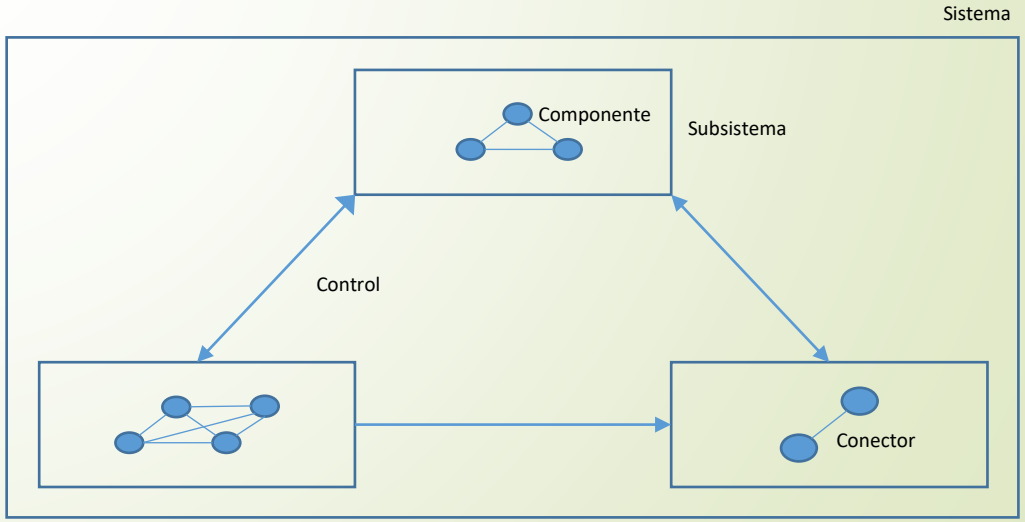
- ¿Porqué es importante la ARQUITECTURA DE SOFTWARE?. Sus razones:
  - Un DISEÑO POBRE produce software defectuoso
  - No existe una planificación adecuada
- ¿Cuándo se conduce un DISEÑO DE SOFTWARE?
  - En una fase temprana del ciclo de vida del desarrollo de software



Arquitectura de software: Un puente entre los requerimientos y la implementación

- Provee una guía para que a partir de los requerimientos se logre la implementación del producto.
- Momento de tomar múltiples decisiones.
- Muestra como los elementos del SISTEMA son estructurados y como trabajan JUNTOS.
- Cubre los requerimientos funcionales y no funcionales.
- Sirve para evaluar el software.

Arquitectura



Definiciones

- IEEE std 1471
  - "Organización de un sistema incorporando elementos, relaciones, ambientes y principios que guían la evolución del diseño"
- Garlan and Shaw
  - "Descripción de elementos que comprimen un sistema conjuntamente con sus relaciones y patrones y los principios que guían su composición y restricciones"

Tareas de un arquitecto de software

- Descomposición del sistema en subsistemas y sus comunicaciones.
- Establece el control dinámico entre las relaciones de los subsistemas.
- Evalúa estilos arquitectónicos para ser considerados como alternativas de solución.
- Realiza un análisis de compensación entre la calidad de los atributos y los requerimientos no funcionales durante la selección de los estilos de arquitectura.

Estilos de arquitectura

- Conocido como “PATRON ARQUITECTÓNICO”.
- Componentes de un estilo de arquitectura:
  - Elementos que desempeñan funciones requeridas por el sistema.
  - Conectores que permiten la comunicación.
  - Restricciones que definen como los elementos pueden ser integrados en el sistema.
  - Atributos que describen las ventajas y desventajas de la estructura escogida.
- Ejemplo: El paradigma de ORIENTACION A OBJETOS

Atributos de calidad

- Atributos de implementación (no se observan en tiempo de ejecución).
  - Interoperabilidad
  - Mantenibilidad y extensibilidad
  - Capacidad de prueba
  - Portabilidad
  - Escalabilidad
  - Flexibilidad

Atributos de calidad

- Atributos en tiempo de ejecución
  - Disponibilidad
  - Seguridad
  - Desempeño
  - Usabilidad
  - Confiabilidad
  - Mantenibilidad
- Atributos de negocio
  - Hora de comprar
  - Costo
  - Tiempo de vida

Guía para diseñar arquitecturas

- Pensar en QUE HACER antes de COMO HACER.
- Pensar en el diseño ABSTRACTO antes de pensar en el diseño ESPECIFICO.
- Pensar en los REQUERIMIENTOS NO FUCIONALES antes que en el DISEÑO DEL PROCESO.
- Pensar en la REUTILIZACION DEL SOFTWARE tanto como sea posible.
- Promocionar la ALTA COHESION y BAJA COHERENCIA.
- Tolerar el refinamiento del diseño.
- Evitar el DISEÑO AMBIGUO y DISEÑO SOBREDETALLADO.



Diseño de arquitectura de software

M.Sc. Percy Huertas Niquén

Introducción

- Detalla los principios de diseño e implementación en proyectos de software.
- Es el soporte de los requerimientos funcionales y no funcionales.
- Permite la heterogeneidad: Un producto de software implementado en diferentes lenguajes de programación, diferentes plataformas o diferentes marcos de trabajo.
- Es importante por el cambio continuo de los requerimientos.

Tipos de estructuras de software

La arquitectura puede expresarse en:

- Unidades de código
- Módulos de software
- Componentes
- Hilos
- Procesos
- Sesiones
- Transacciones
- Objetos

Estructura estática de software

- Todo software es implementado en muchos archivos que incluyen archivos estáticos, ejecutables, librerías, código binario, DLL, Javabeans, etc.
- En tiempo de desarrollo, los elementos principales con el código fuente de los módulos.
- Ambos elementos deben tener conectores, los cuales deben tener los siguientes atributos:
  - Dirección: conexión entre módulos
  - Sincronización
  - Secuencia
- Ejemplo: estructura de java (implementación de clases, atributos, conexiones, etc.)

Estructura del software en tiempo de ejecución

- Análisis del código y como se ejecuta.
- El código debe contemplar:
  - Multiplicidad
  - Distancia y conexión media
  - Universalidad en la invocación
  - Autodescriptivo

Administración de la estructura de software

- Un gran proyecto es ejecutado por un conjunto de equipos
- Cada equipo realiza la manipulación adecuada del módulo. Se debe tener presente las dificultades de conectividad
- Una buena arquitectura permite:
  - Flexibilidad es estos detalles
  - Flexibilidad en la conectividad
  - Flexibilidad en la comunicación

Elementos a tomar en cuenta para el software

- Si un elemento es reentrante entonces debe ser implementado por un HILO o por un PROCESO
- Si un elemento no es reentrante y mucho HILOS o PROCESOS necesitan de la comunicación entonces debes ejecutarse en HILOS separados
- Si un elemento tiene alta multiplicidad y su desempeño es importante en el sistema a construir; un SERVICIO debe implementar los elementos
- Si los elementos tienen cálculos pesados entonces se debe emplear un cluster de procesadores
- Un elemento complejo puede ser expandido en los subsistemas

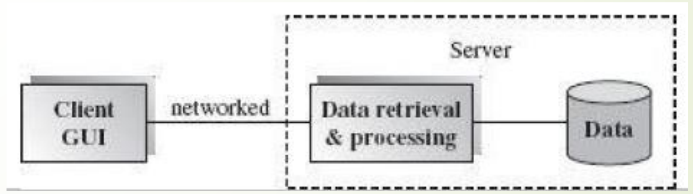
Propuesta ágil en el diseño arquitectónico

- La arquitectura de software tradicional emplea modelos clásicos
- Un método ágil permite la reutilización de manera adecuada según criterios expertos del diseñador
- Para desarrollar un producto se puede visualizar una MODULACION del mismo lo que permite que la arquitectura también se puede MODULARIZAR

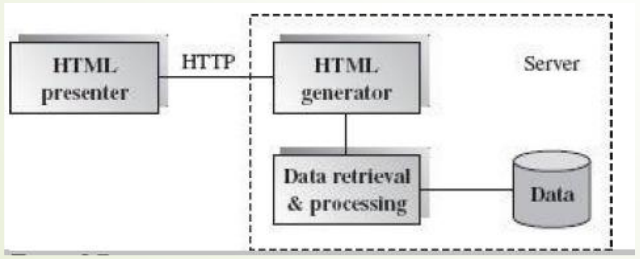
1



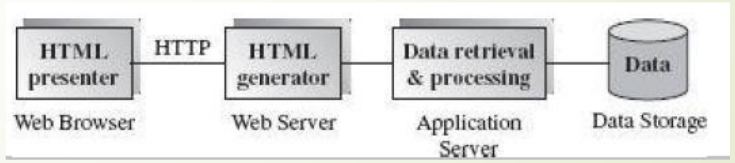
2

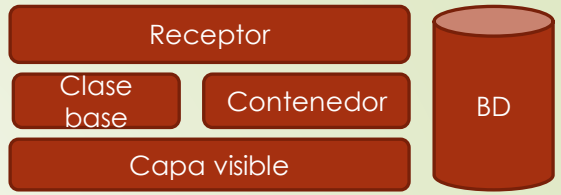


3



4





Modelos de arquitectura de software

M.Sc. Percy Huertas Niquén

Introducción

- Arquitectura de software especifica la abstracción de un sistema software
- Para esto se emplea:
  - Descomposición
  - Estilos arquitectónicos
  - Atributos de calidad

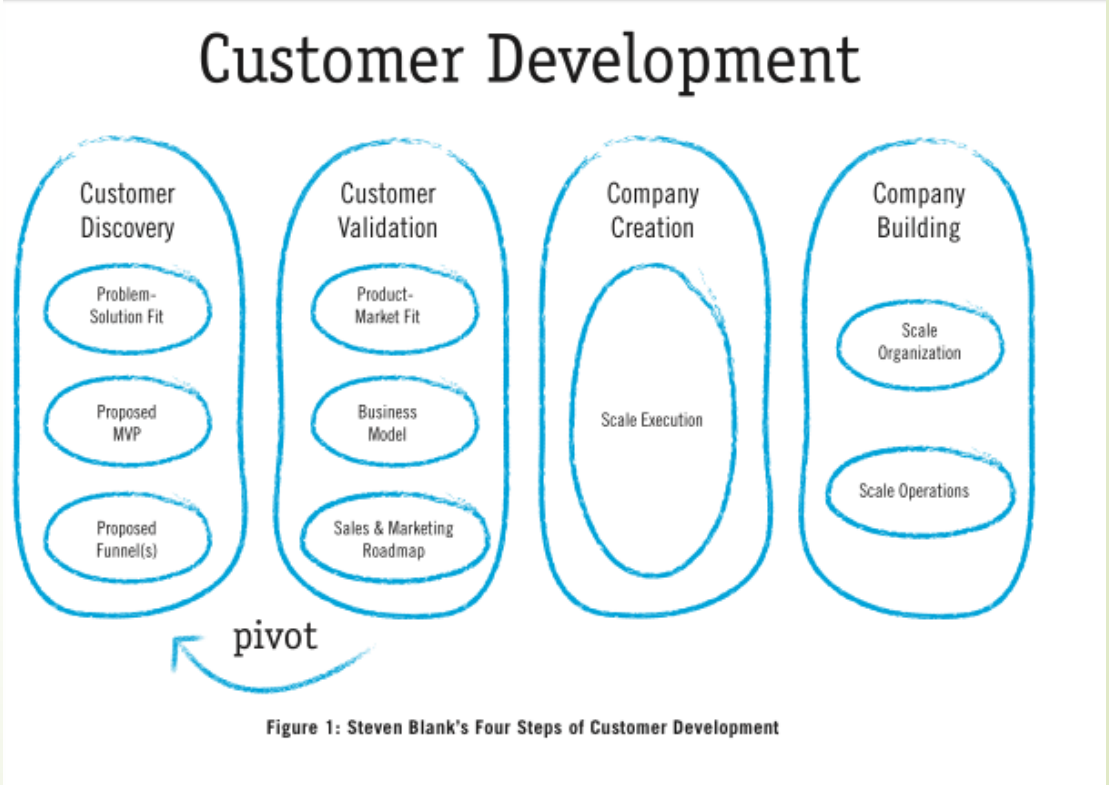


Figure 1: Steven Blank's Four Steps of Customer Development

UML para arquitectura de software (parte estática)

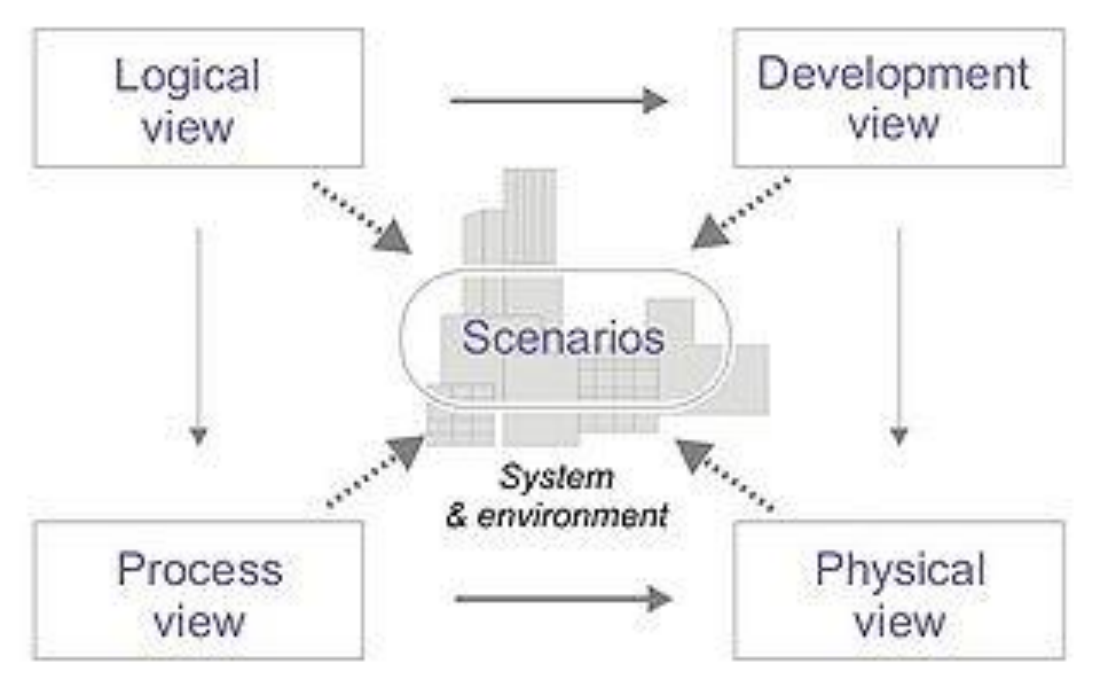
- Diagrama de clase
- Diagrama de objetos
- Diagrama de estructura
- Diagrama de componentes
- Diagrama de paquetes
- Diagrama de despliegue

UML para arquitectura de software (parte dinámica)

- Diagrama de casos de uso
- Diagrama de actividades
- Diagrama de estados
- Diagrama de secuencias
- Diagrama de interacción
- Diagrama de comunicación
- Diagrama de tiempos (visto en UML 2.0)

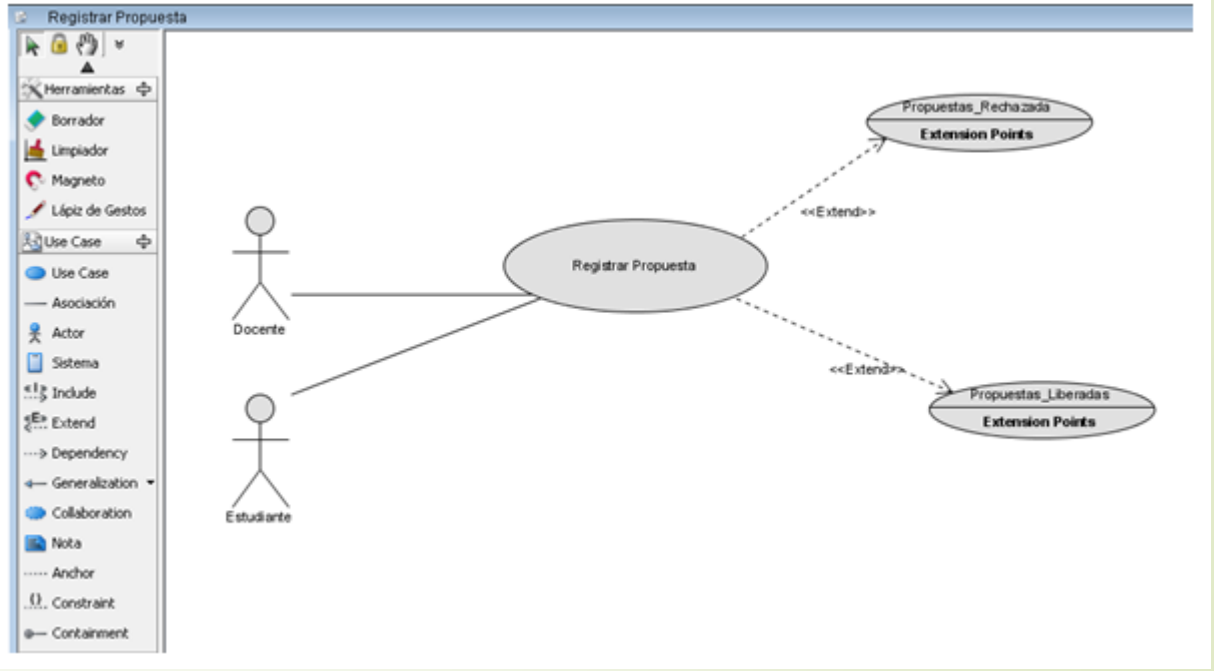
Arquitectura de los modelos de vista

- Se emplea el modelo 4+1



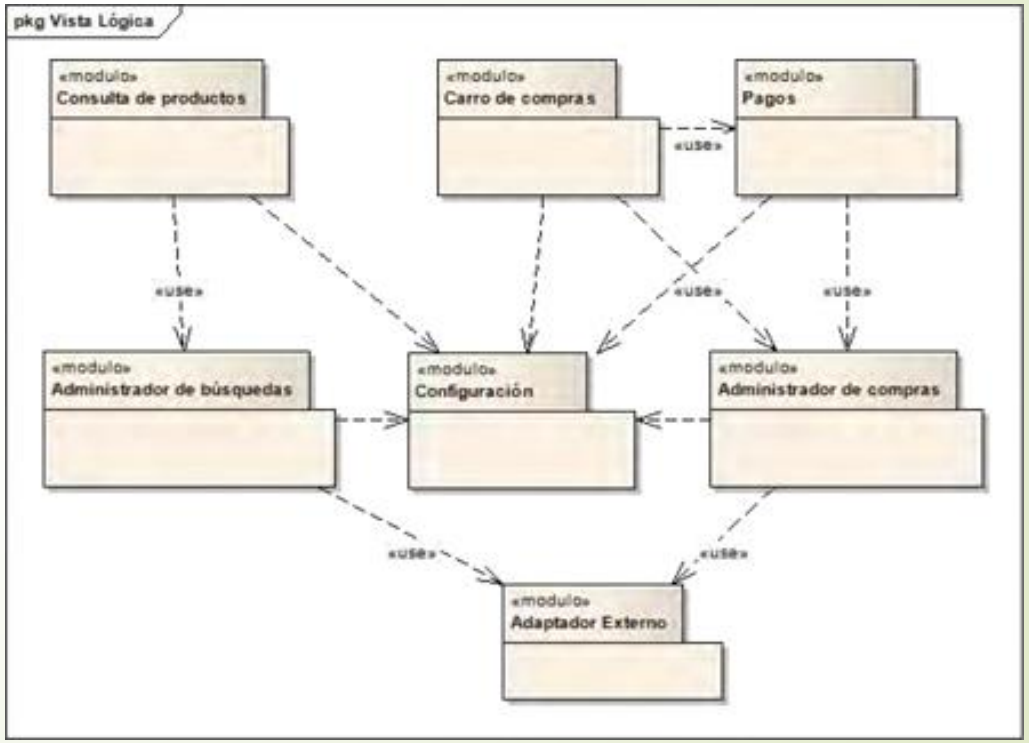
Vista de escenario

- Describe la funcionalidad del sistema
- Se emplean los casos de uso



Vista lógica o conceptual

- Basada en el dominio de la aplicación, requisito indispensable para la implementación de requerimientos funcionales



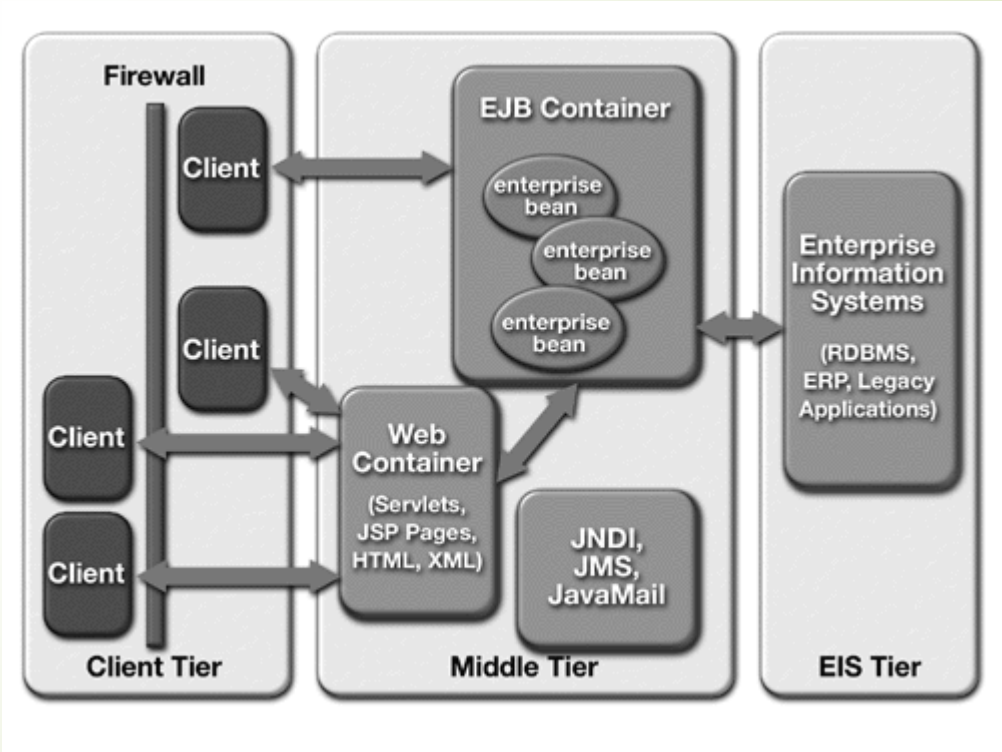
Vista de desarrollo

- Deriva de la vista lógica
- Describe es aspecto ESTATICO de los módulos del sistema



Vista de procesos

- Trata los aspectos dinámicos del sistema
- Este vista se deriva de la vista lógica



Vista física

- Describe la instalación, configuración y desarrollo de una aplicación
- Muestra un mapeo entre el software y hardware

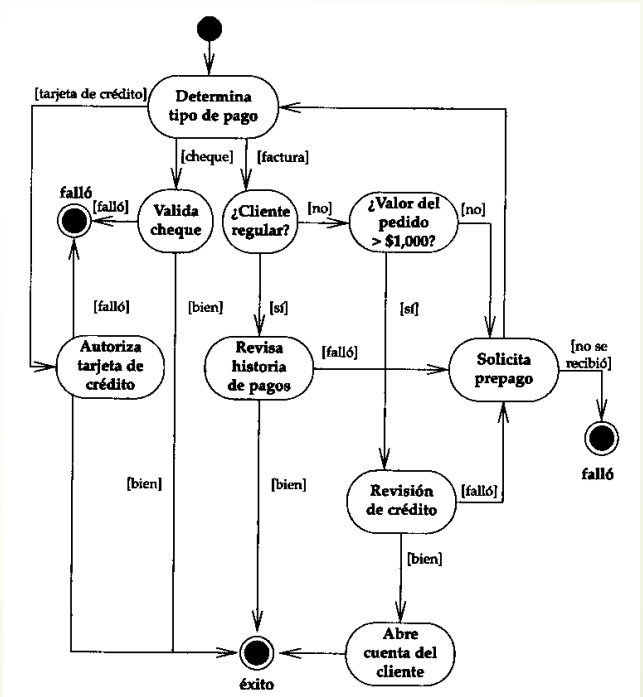
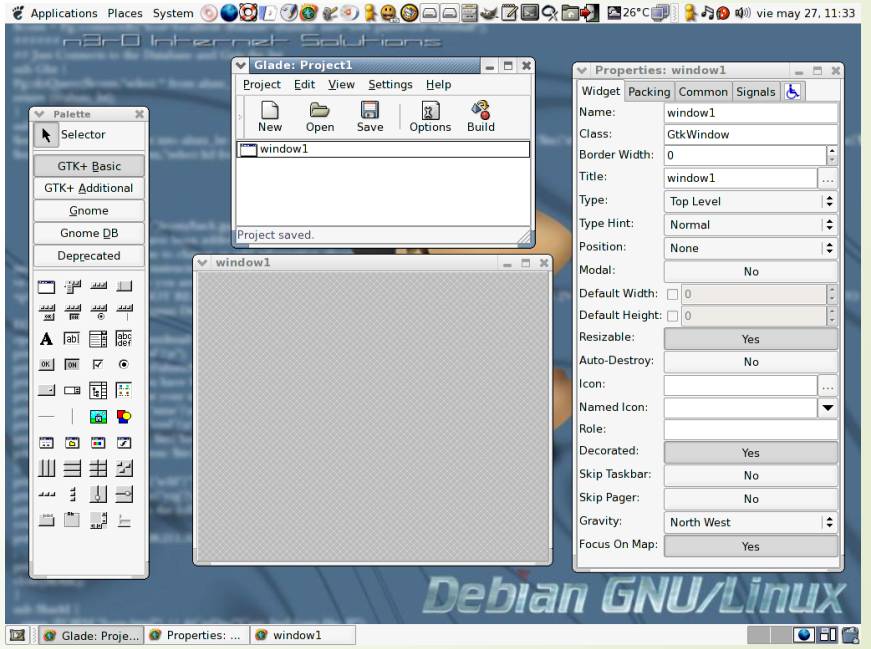


Figura 9-6: *Diagrama descompuesto de actividades*

Vista de interfaces de usuario

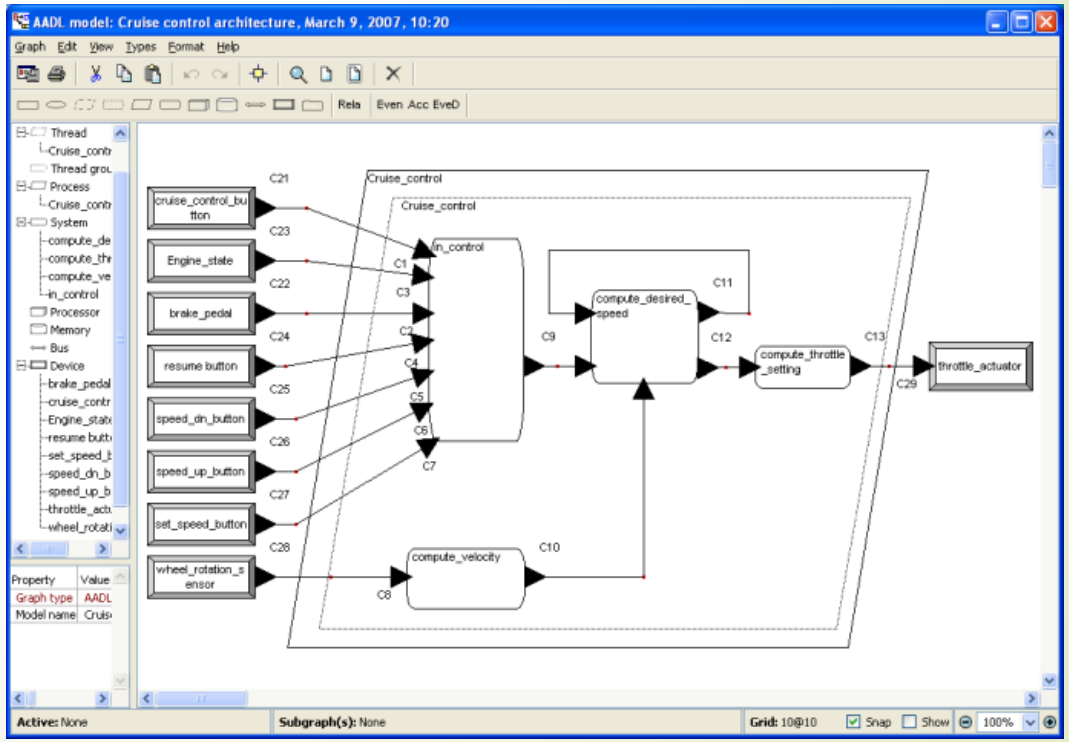
- Vista que provee interfaces de usuario claros y detalles de la implementación
- Muestra el modelo de navegación bajo un prototipo

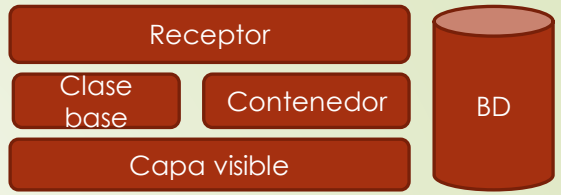


Lenguaje de descripción de la arquitectura (ADL)

Para aplicar el ADL se necesita los siguientes requerimientos:

- Composición
- Abstracción
- Reutilización
- Configuración
- Heterogeneidad
- Análisis

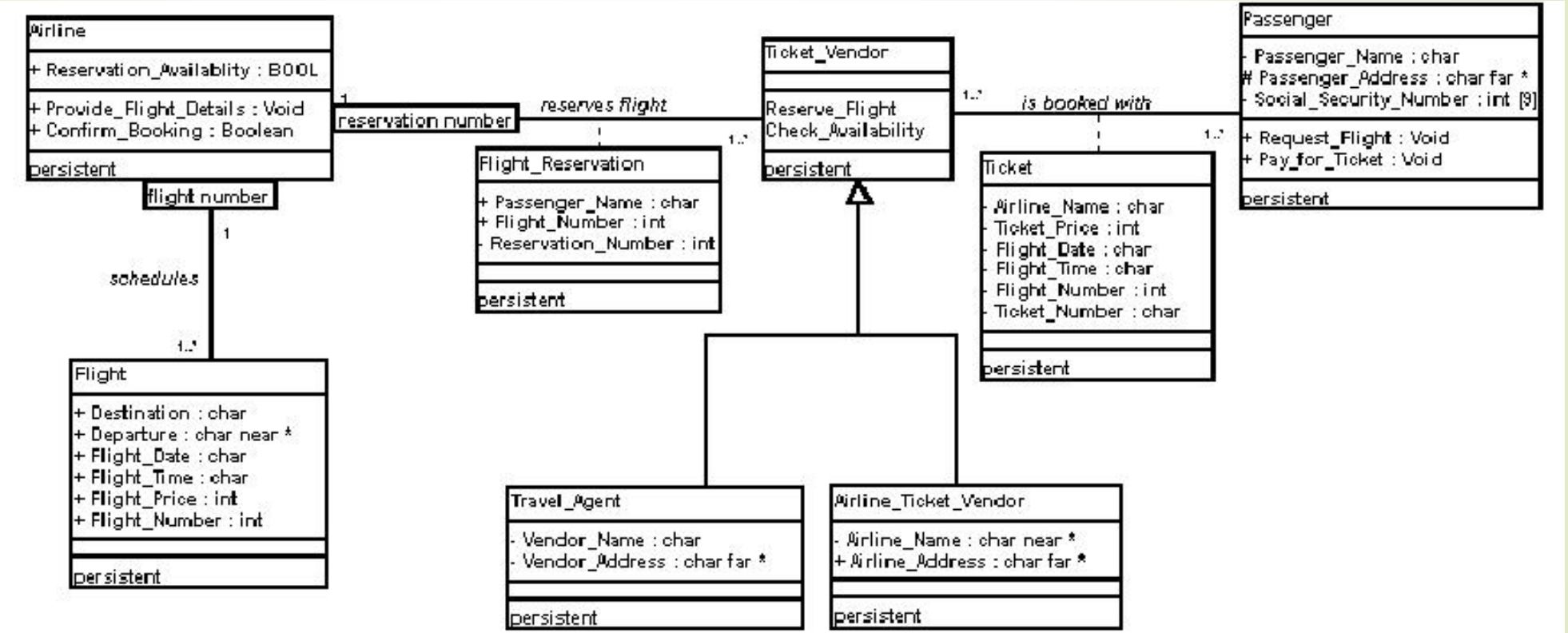




Parte 3: Diseño de software orientado a objetos

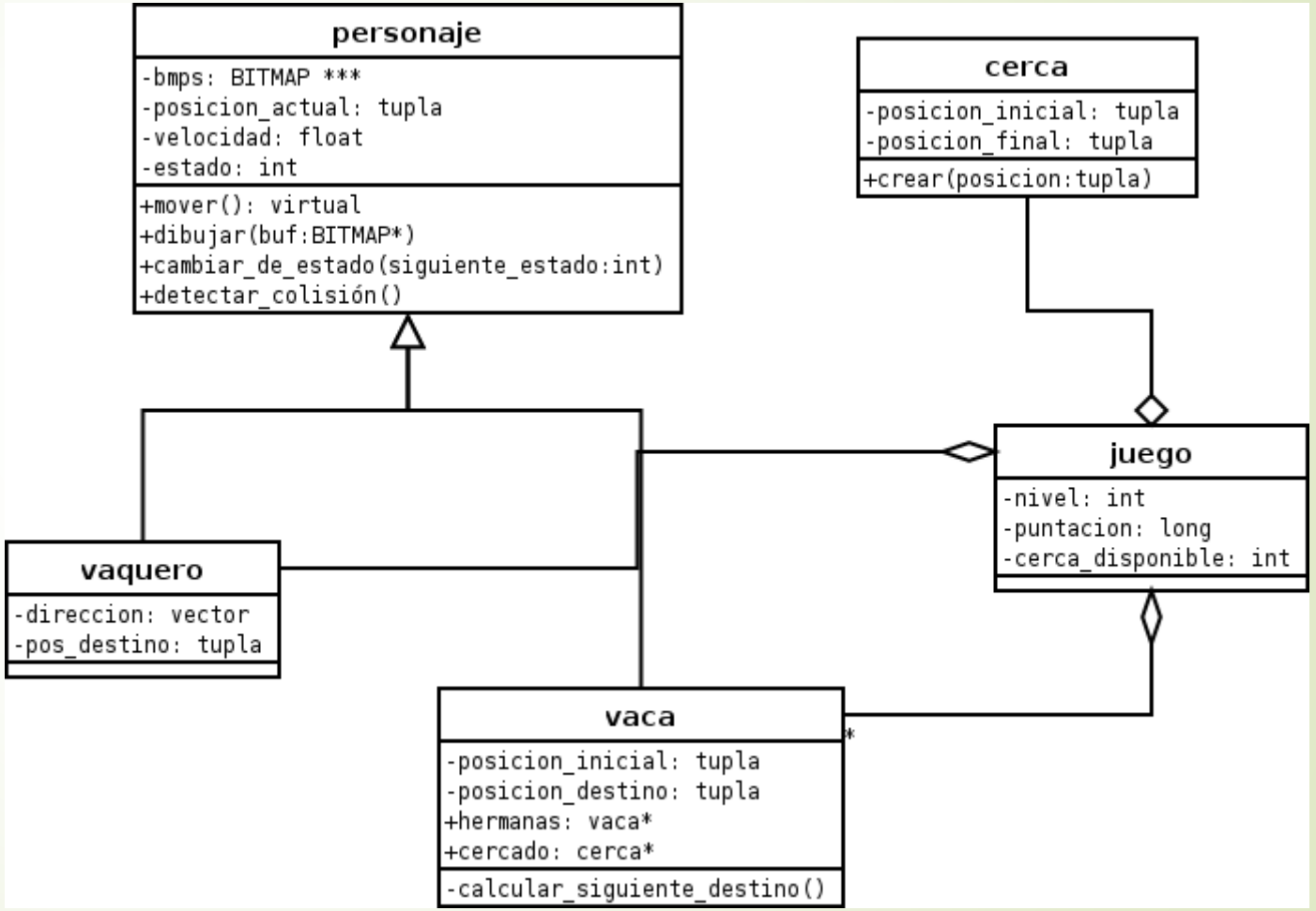
M.Sc. Percy Huertas Niquén

Clases


Airline
+ Reservation_Availablity : BOOL
+ Provide_Flight_Details : Void
+ Confirm_Booking : Boolean
persistent
1
reservation number
reserves flight
1..*
Flight_Reservation
+ Passenger_Name : char
+ Flight_Number : int
- Reservation_Number : int
persistent
Ticket_Vendor
Reserve_Flight
Check_Availability
persistent
1..*
is booked with
1..*
Passenger
- Passenger_Name : char
Passenger_Address : char far *
- Social_Security_Number : int [9]
+ Request_Flight : Void
+ Pay_for_Ticket : Void
persistent
Ticket
- Airline_Name : char
- Ticket_Price : int
- Flight_Date : char
- Flight_Time : char
- Flight_Number : int
- Ticket_Number : char
persistent
flight number
1
schedules
1..*
Flight
+ Destination : char
+ Departure : char near *
+ Flight_Date : char
+ Flight_Time : char
+ Flight_Price : int
+ Flight_Number : int
persistent
Travel_Agent
- Vendor_Name : char
- Vendor_Address : char far *
persistent
Airline_Ticket_Vendor
- Airline_Name : char near *
+ Airline_Address : char far *
persistent

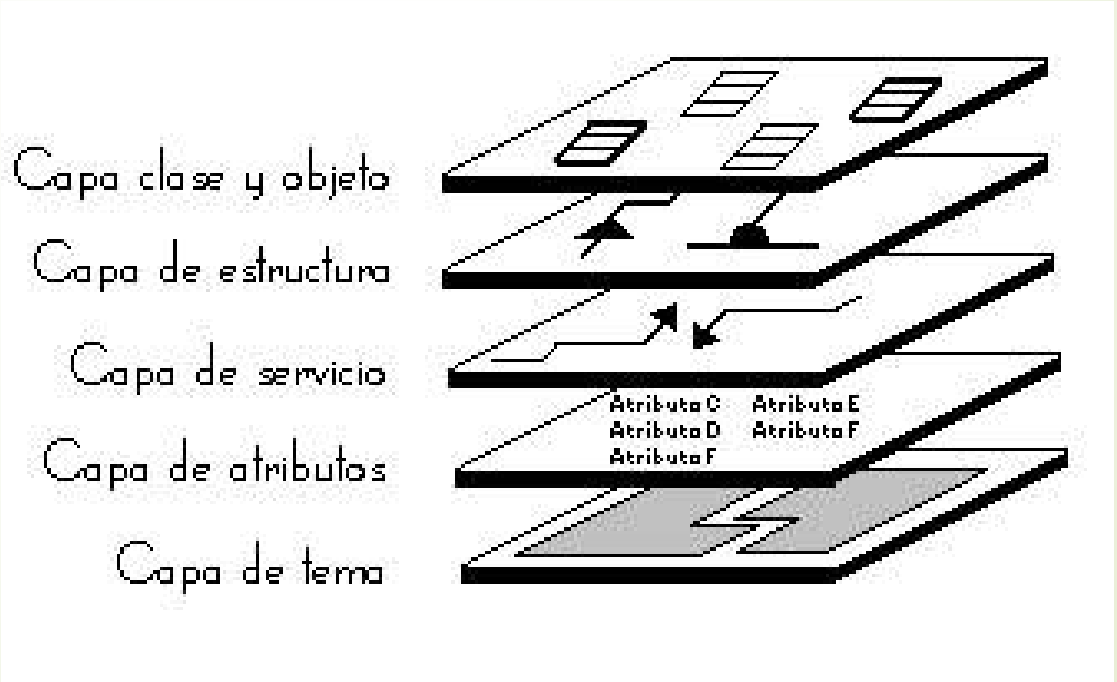
Relaciones

- Composición
- Agregación
- Asociación
- Herencia

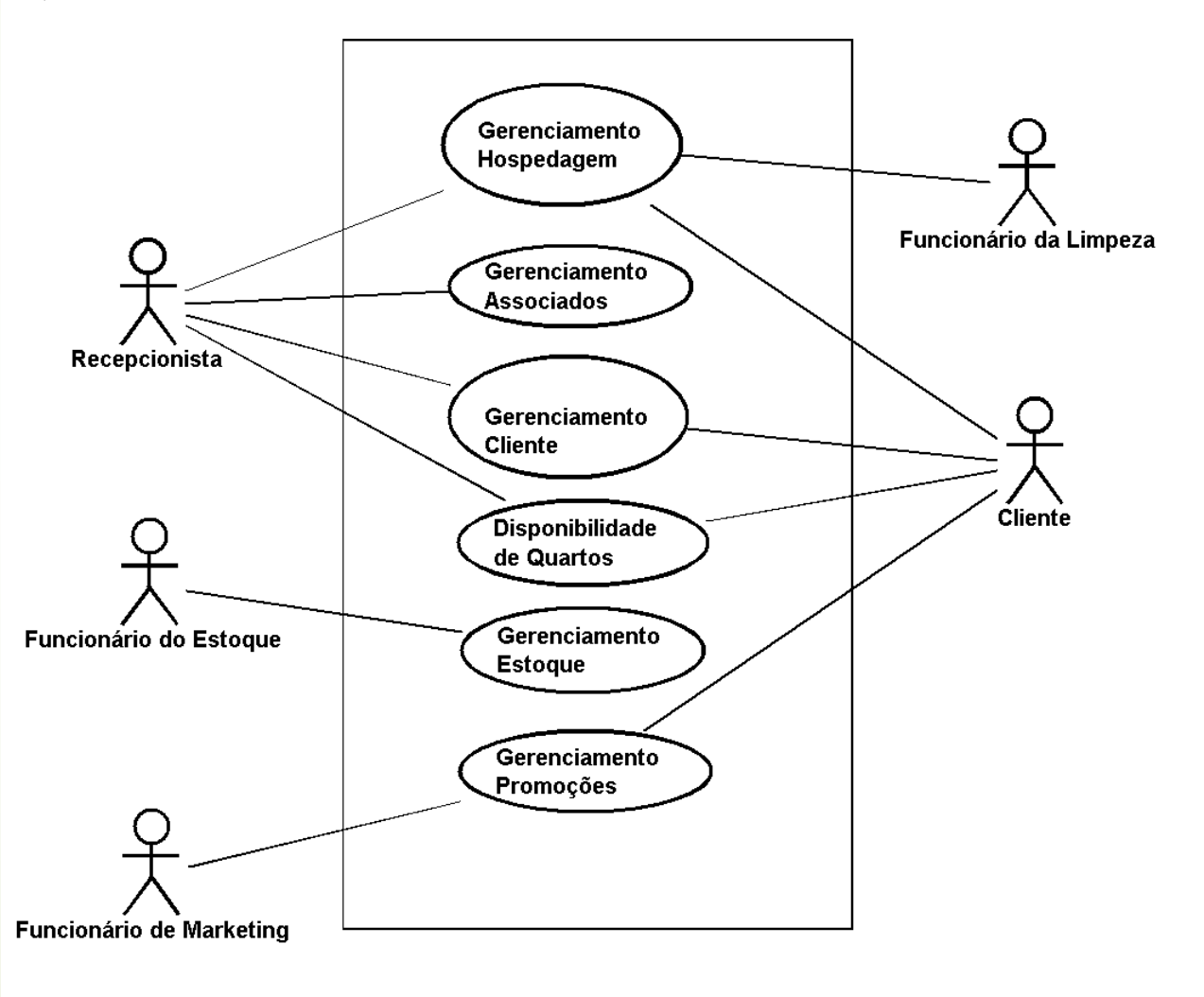


Análisis orientado a objetos

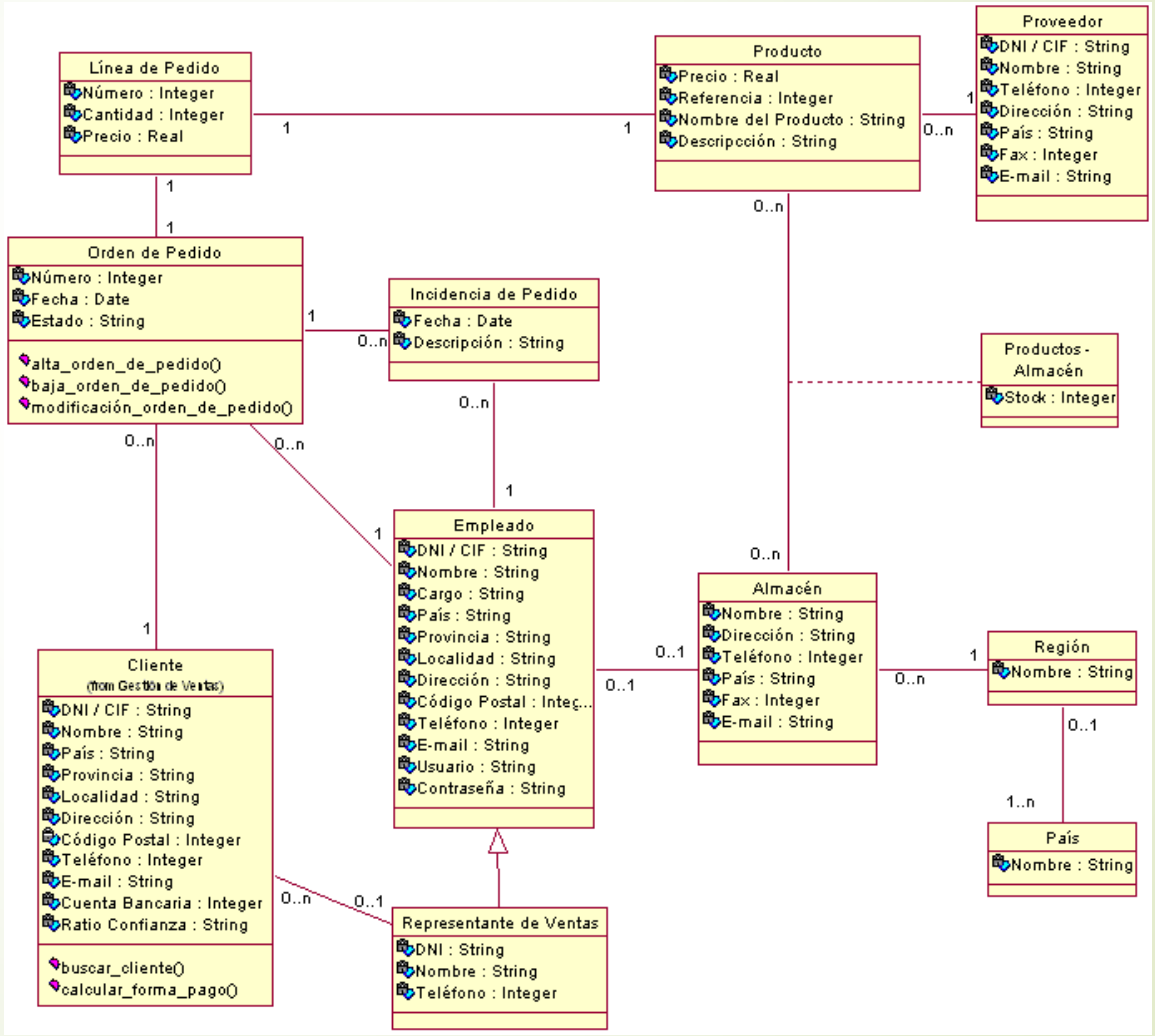
- Desarrolla los procesos en tres pasos: Análisis orientados a objetos, diseño orientado a objetos e implementación orientada a objetos



Paso 1: Diseño por casos de uso

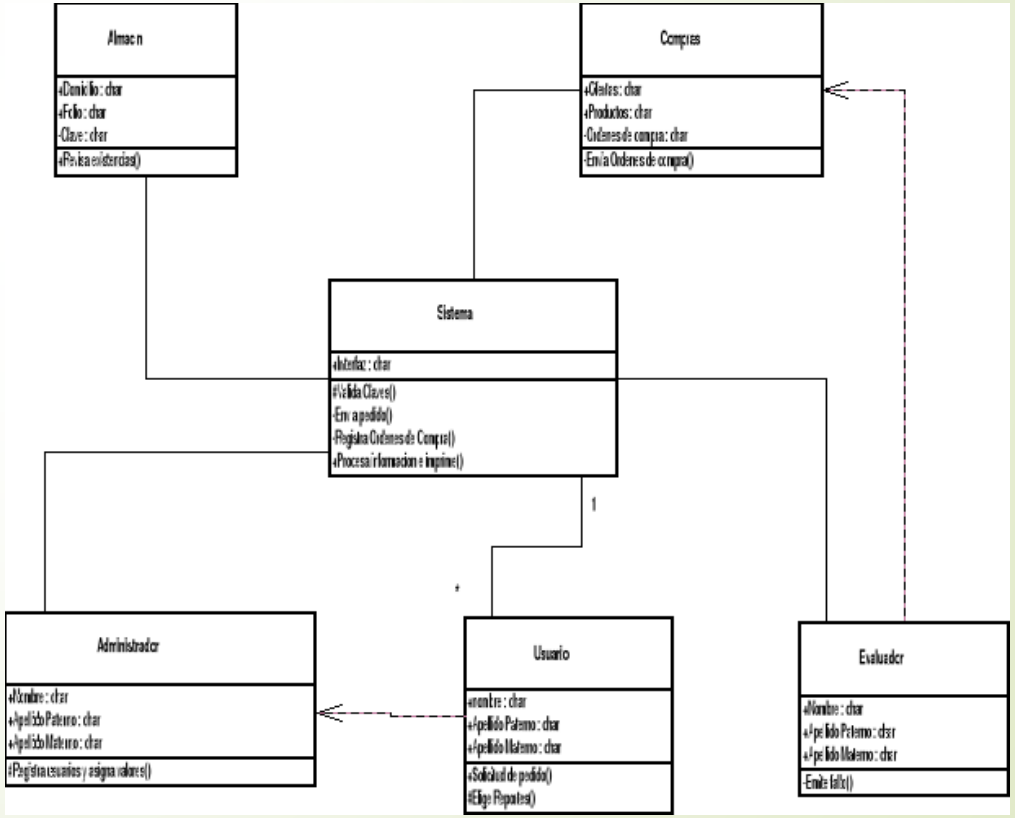


Paso 2: Desarrollo de las clases de análisis



Diseño orientado a objetos

- Desarrolla la arquitectura estructural de un sistema



Paso 1: Identificación de clases

Tabla 2.19 Tarjeta CRC Módulo At1

Módulo At1	
Funcionalidades	**Colaboraciones**
Insertar At1 Modificar At1 Eliminar At1 Mostrar At1 Buscar At1 Exportar At1	Operador Clave Cierre Mantenimiento Orden de Servicio Material Empleado

Tabla 2.20 Tarjeta CRC Módulo Operador

Módulo Operador	
Funcionalidades	**Colaboraciones**
Insertar Operador Modificar Operador Eliminar Operador Mostrar Operador	

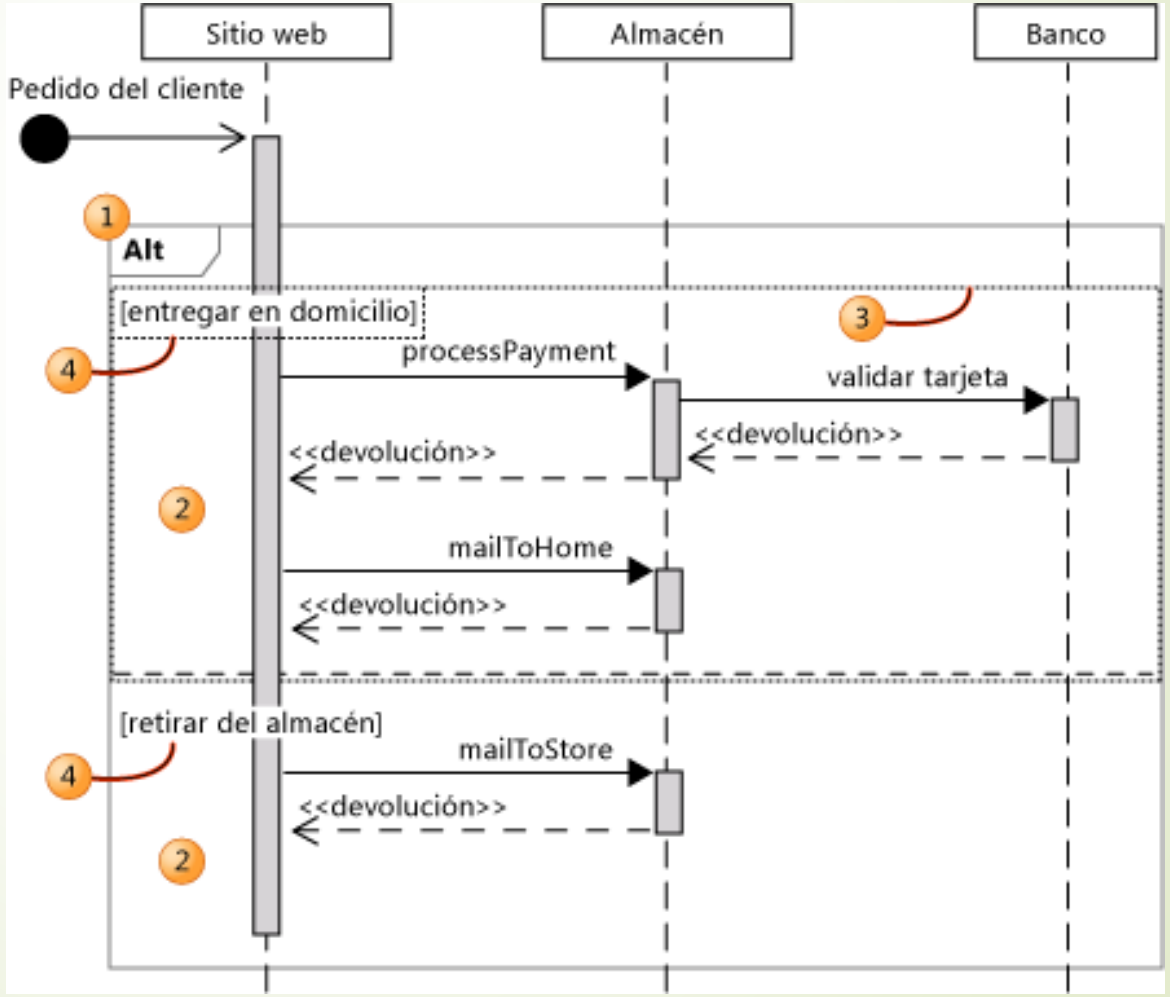
Tabla 2.21 Tarjeta CRC Módulo Clave Cierre

Módulo Clave Cierre	
Funcionalidades	**Colaboraciones**
Insertar Clave Cierre Modificar Clave Cierre Eliminar Clave Cierre Mostrar Clave Cierre	

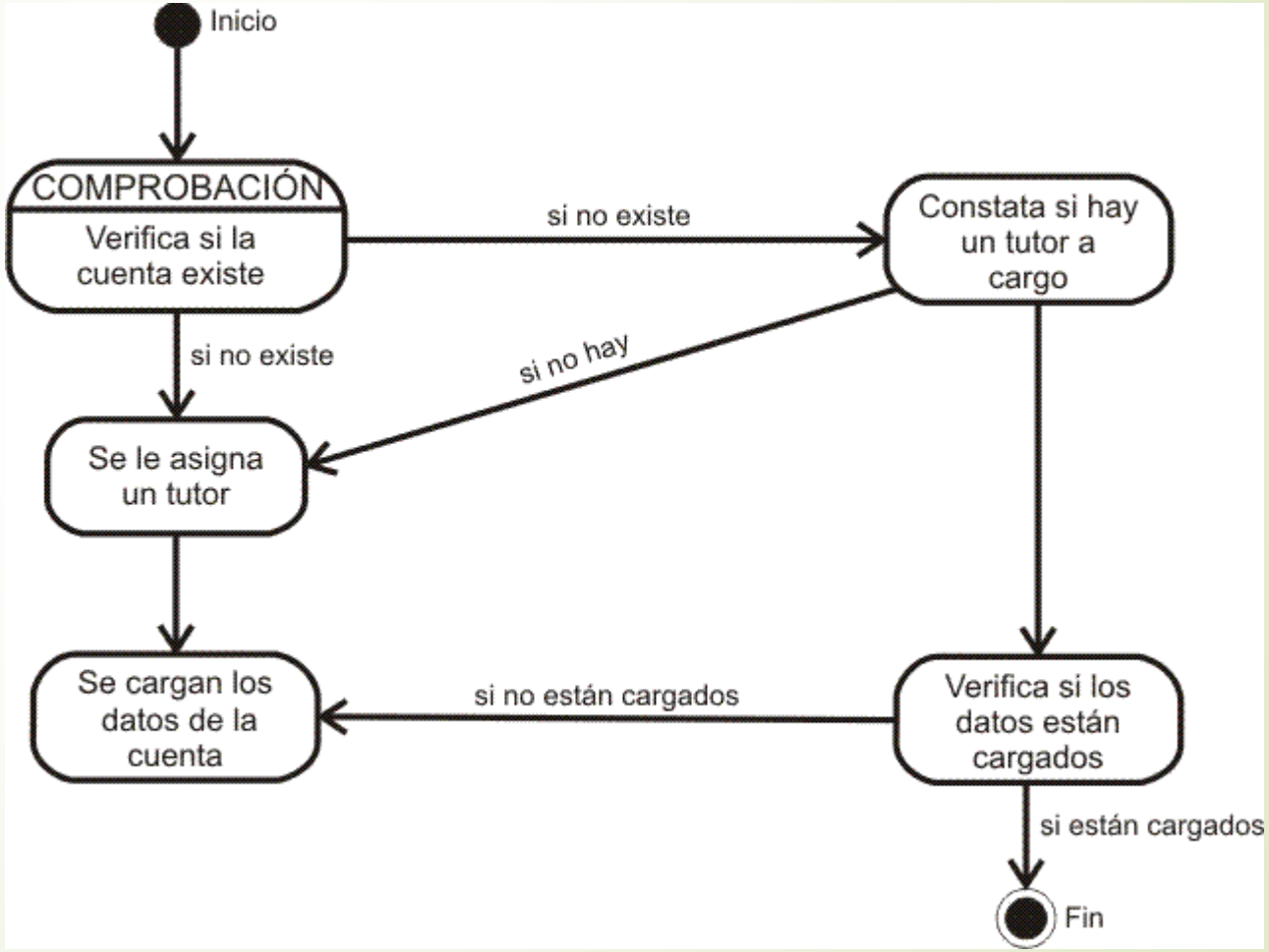
Tabla 2.22 Tarjeta CRC Módulo Material

Módulo Material	
Funcionalidades	**Colaboraciones**
Insertar Material Modificar Material Eliminar Material Mostrar Material	

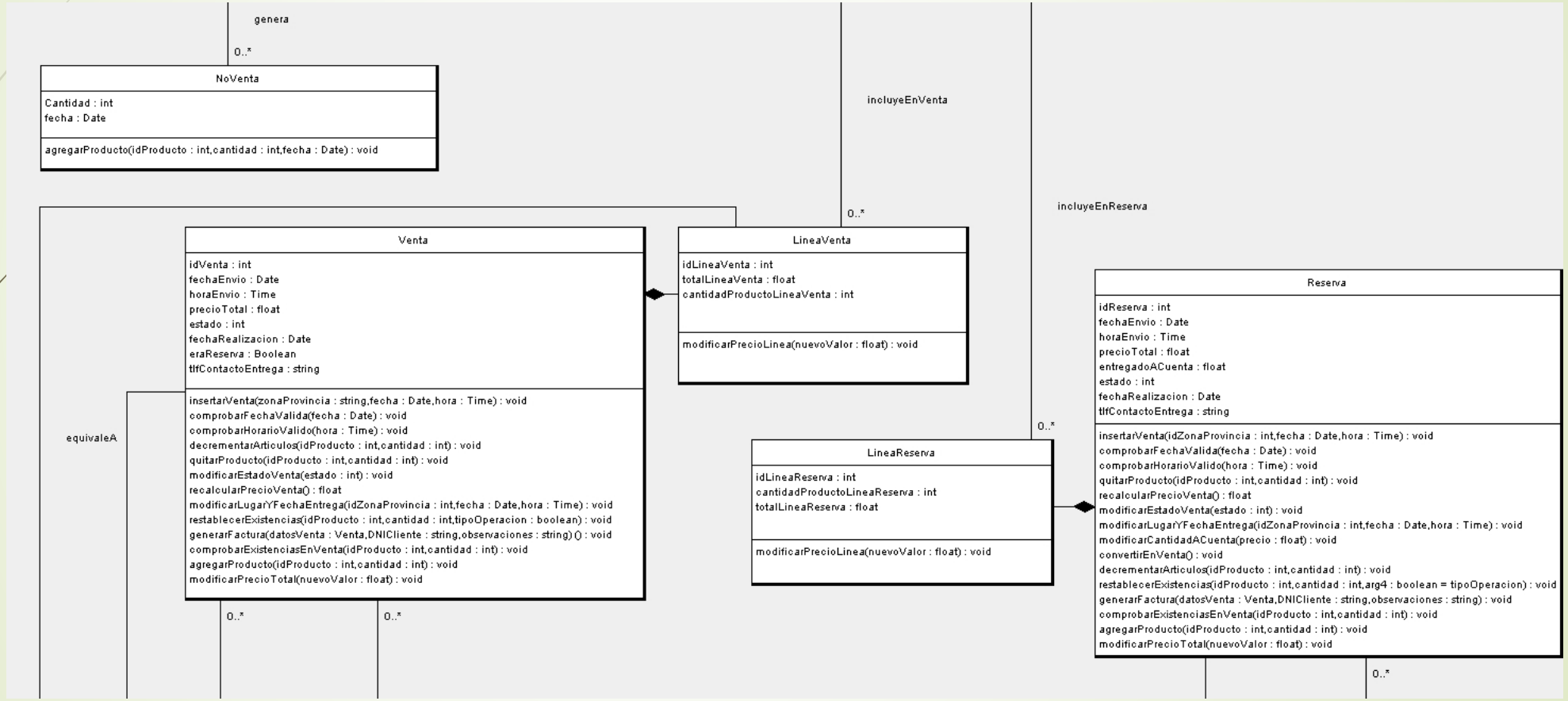
Paso 2: Construir el diagrama de interacción



Paso 3: Construir el diagrama de estados



Paso 4: Especificación de clases





Parte 4: Estilos de arquitectura

M.Sc. Percy Huertas Niquén



Arquitectura orientada al flujo de datos

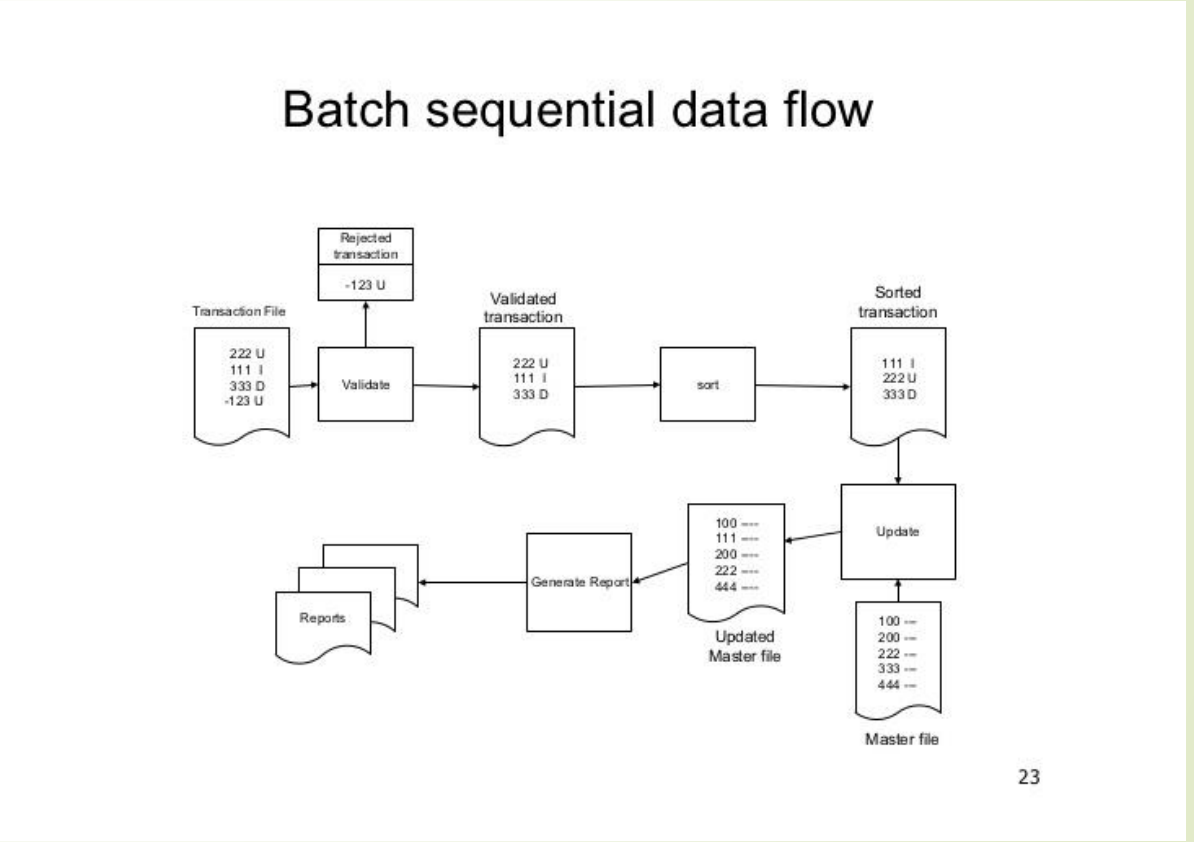
M.Sc. Percy Huertas Niquén

Introducción

- Mira al sistema como la transformación de un conjunto de datos
- Datos y operaciones son independientes
- Cada componente presenta una entrada y salida de datos
- En la práctica la conexión se hace como:
  - I/O streams
  - I/O files
  - Buffers

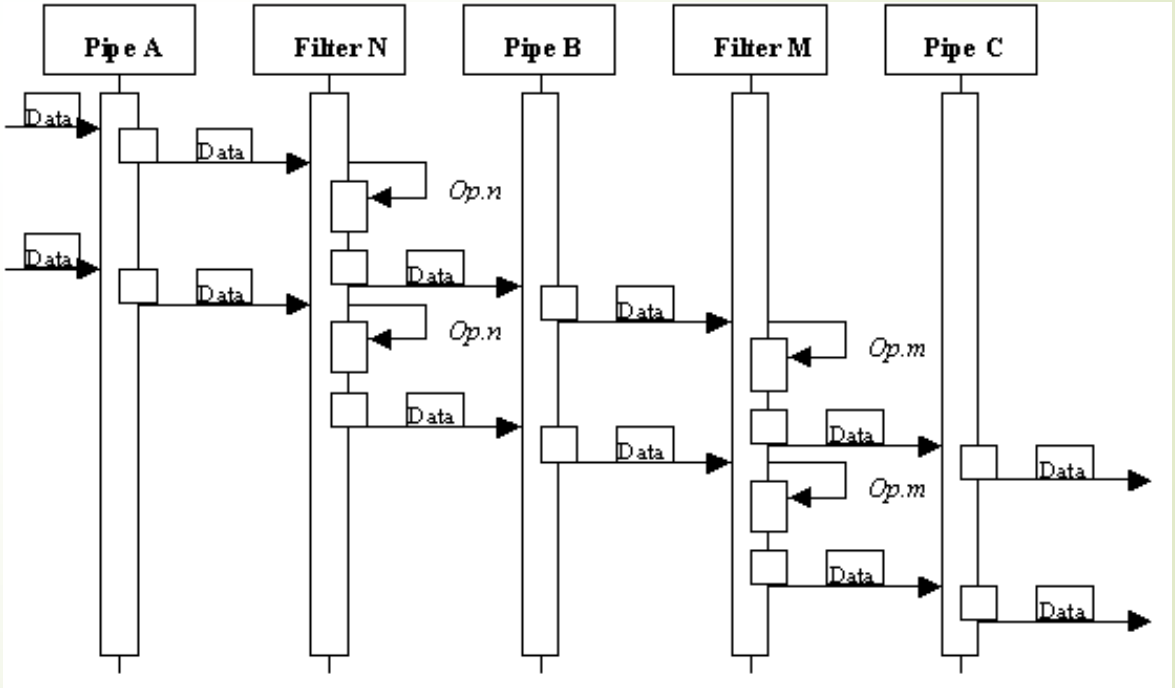
Lotes secuenciales

- Modelo tradicional



Arquitectura por tuberías y filtros

- Descompone el sistema en componentes de fuentes de código, filtros, tuberías y sumideros de datos



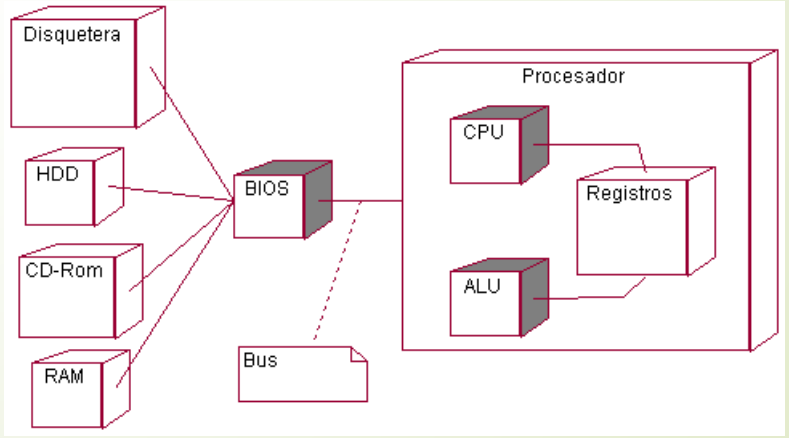


Arquitectura de software centrada en el dato

M.Sc. Percy Huertas Niquén

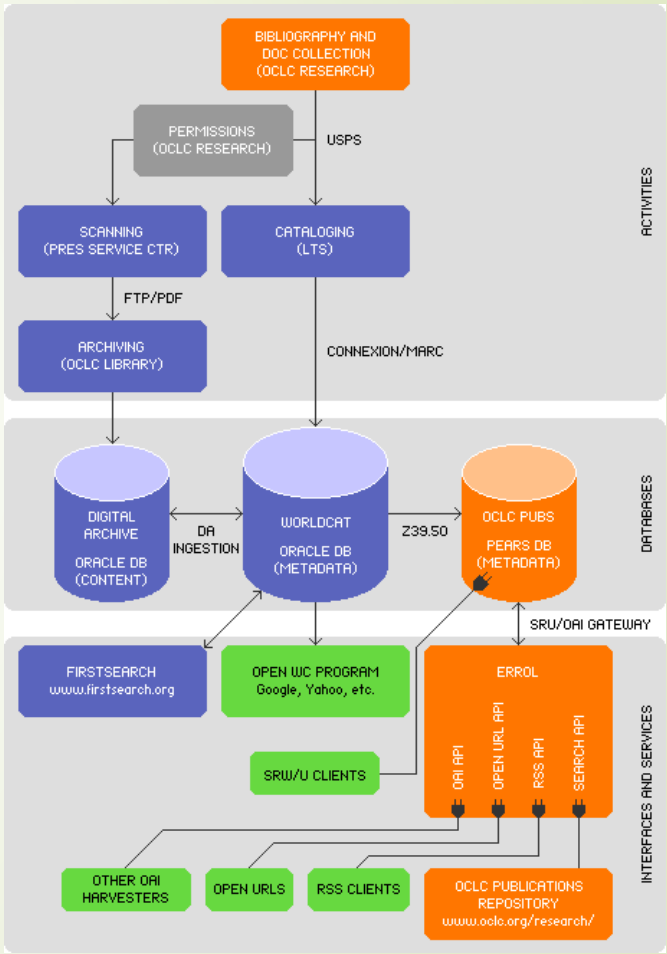
Introducción

- Se caracteriza por un almacenamiento centralizado de datos
- Es compartido por todo tipo de software de la organización
- El sistema es descompuesto en:
  - Almacenamiento de datos
  - Agentes
- Toda la comunicación entre módulos es vía datos



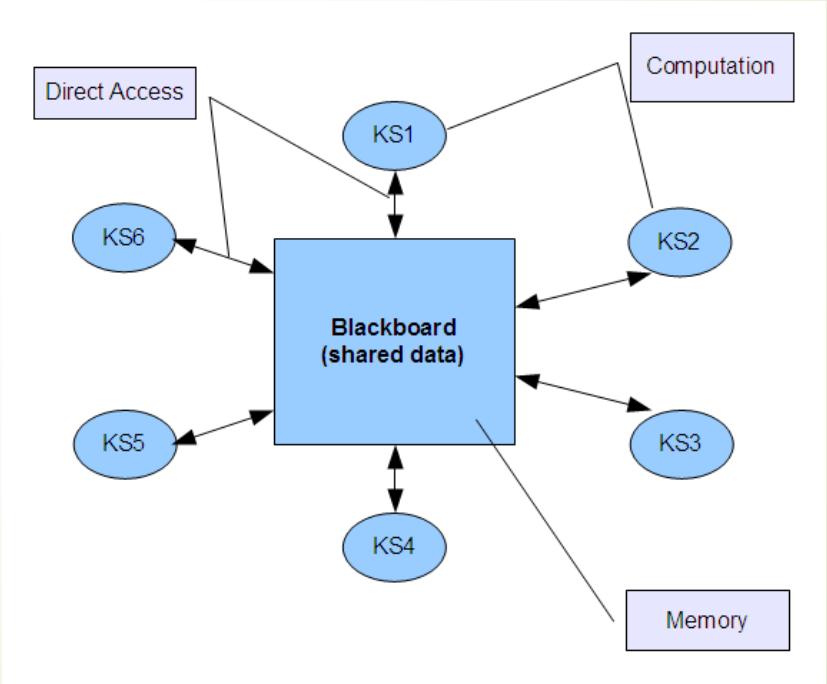
Repositorio de estilos de arquitectura

- Soporta el procesamiento de datos por el usuario



Arquitectura de pizarra

- Desarrollado para sistemas que poseen interfaces para el habla, patrones de reconocimiento, inteligencia artificial, compiladores
- La idea es similar a enseñar en una clase universitaria



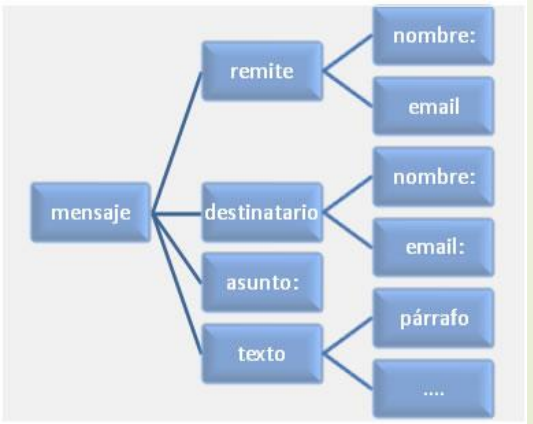


Arquitectura jerárquica

M.Sc. Percy Huertas Niquén

Introducción

- El sistema se observa como una estructura jerárquica
- El software es descompuesto en subsistemas lógicos con diferentes niveles en la jerarquía
- Los módulos en diferentes niveles son invocados por métodos explícitos
- En algunos lenguajes de programación primero se deben de incluir las librerías para obtener funcionalidad
- En una implementación orientada a objetos los servicios son organizados en paquetes



Subrutina principal

- El principal objetivo es permitir la reutilización
- Un caso típico es la programación en C++

```
#include<stdio.h>
#include<math.h>

int main(void){
    float a,b,c;
    float e=2.7182;
    float fun1,fun2;

    printf("\tLa funcion es : (e^-x)-x\n");
    printf("Ingrese el primer valor: ");
    scanf("%f", &a);
    printf("Ingrese el segundo valor: ");
    scanf("%f", &b);

    fun1=pow(e,-a)-a;
    fun2=pow(e,-b)-b;

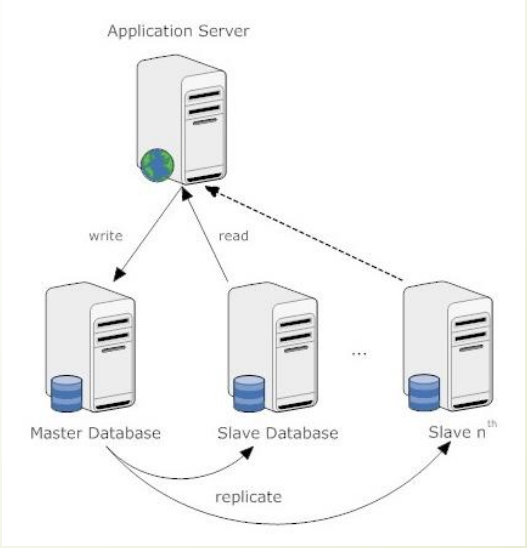
    c=(a)-(fun1*(a-b))/((fun1)-(fun2));


         printf("El valor de c es %f\n ", c);

    getch();
    return 0;
```

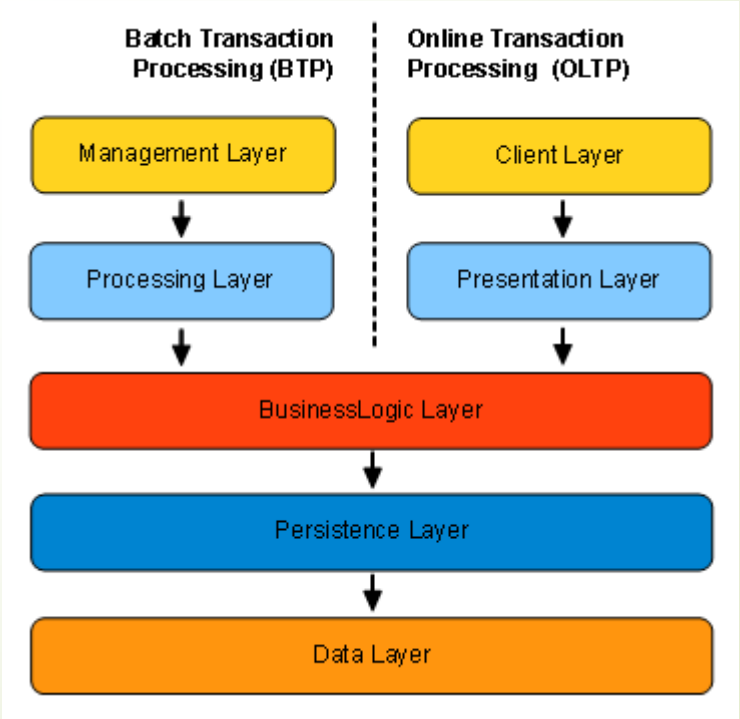
Maestro esclavo

- Es una variante de la arquitectura anterior
- Soporta tolerancia a fallos y sistemas confiables
- Los esclavos proveen servicios al maestro y el maestro selecciona un resultado particular bajo ciertas estrategias



Capas

- El sistema es descompuesto en un determinado número de capas mostrando una jerarquía
- Cada capa consiste en un conjunto de clases que son encapsulados en paquetes





Arquitectura de software para comunicación asíncrona

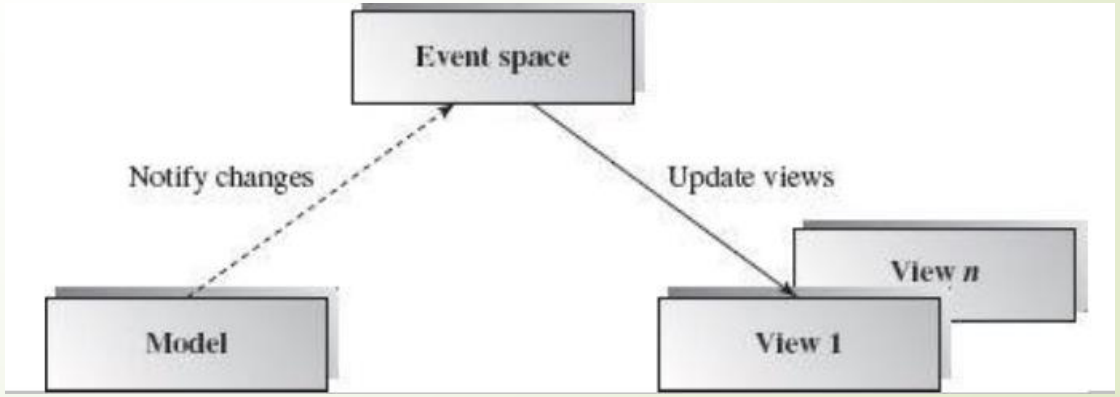
M.Sc. Percy Huertas Niquén

Introducción

- Se emplea cuando se emplea el METODO DE LA PIZARRA
- Puede ser especificado en las dos modalidades:
  - Modalidad del BUFFER
  - Modalidad del NO USAR BUFFER
- Resuelve los problemas de:
  - Productor – consumidor
  - Editor - productor
- Problemas suscitados por patrones
- El objetivo es proveer un desacoplamiento entre los eventos y los mensajes
- Sucede cuando se emplea: JavaBeans, Active X, componentes .NET, C++

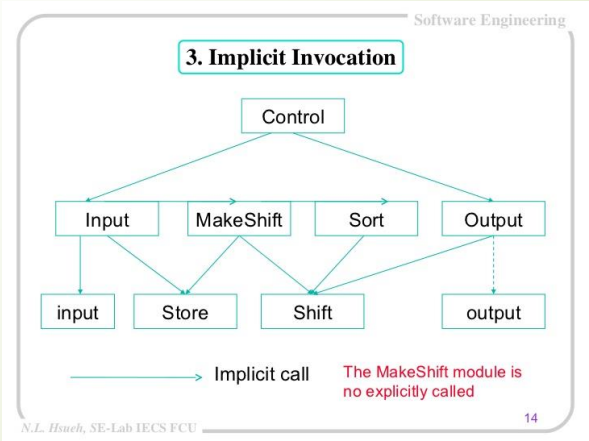
Invocaciones implícitas: Eventos sin buffer

- Quiebra el sistema en dos partes:
  - Fuente de los eventos
  - Escuchadores de eventos
- Son los procesos quienes conectan estas dos partes
- El lenguaje SMALLTALK es muy adicto a estos detalles



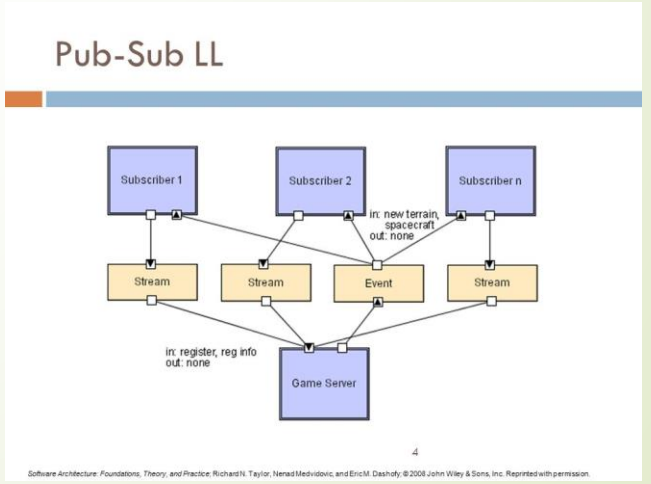
Aplicabilidad

- Componentes gráficos interactivos
- Ambientes integrados o IDE's
- Aplicaciones con un bajo acoplamiento entre componentes que necesitan comunicarse con otros componentes
- Implementación de máquinas de estado
- Cuando los eventos en una aplicación no son predecibles



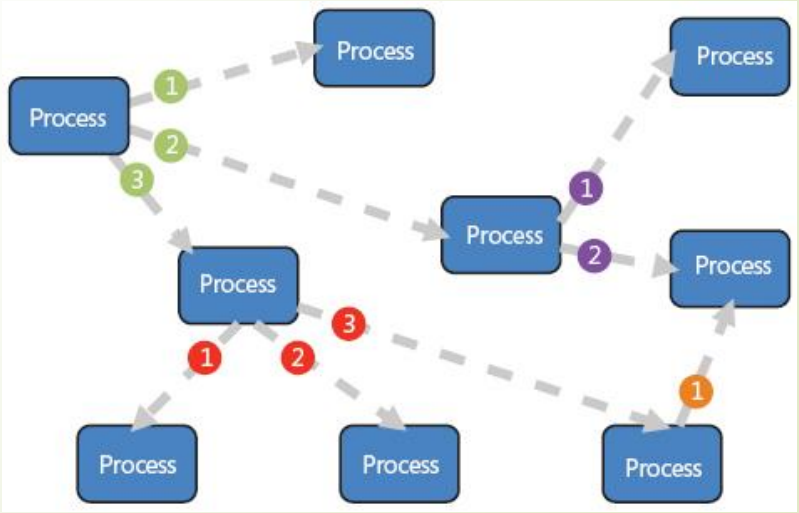
Beneficios

- Disponibilidad de marcos e trabajo. Ejemplo Java AWT y SWING
- Reutilización de componentes
- Mantenimiento y evolución del sistema
- Independencia y flexibilidad en la conectividad
- Ejecución paralela de eventos



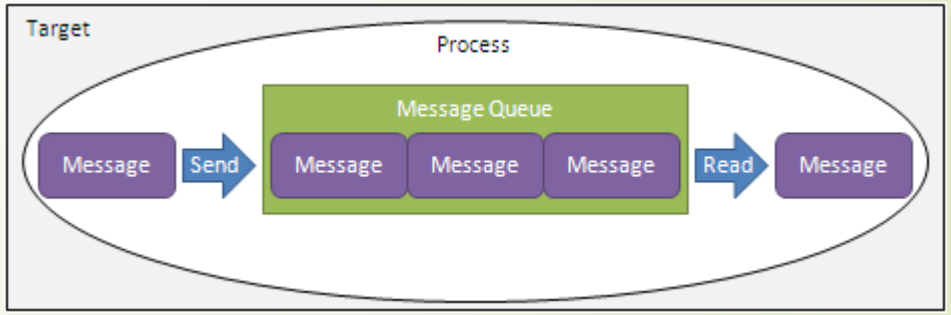
Limitaciones

- Dificultad en hacer un DEBUG del sistema
- Dificultad para probar el sistema
- Existe un acoplamiento más apretado entre eventos
- Poca confianza en la invocación indirecta



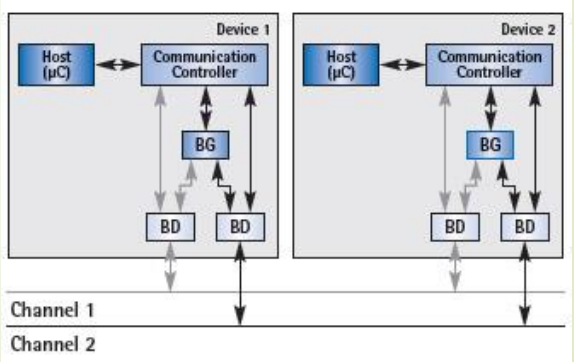
Arquitectura basada en mensajes con BUFFER

- Quiebra el sistema en tres partes:
  - Productores de mensajes
  - Consumidores de mensajes
  - Proveedores de servicios de mensajes
- Todos se interconectan por la COLA DE MENSAJES
- También es considerada como CENTRADO EN EL DATO
- Usado por: Unix, Microsoft, IBM



Aplicaciones

- Bueno para sistemas que implementan el algoritmo del PRODUCTOR – CONSUMIDOR
- El proveedor necesita componentes que funcionen independientemente de la información de otros componentes
- El proveedor necesita que los componentes no se encuentren ejecutando todos al mismo tiempo
- Las aplicaciones comerciales permiten componentes que envían información y continúen operando sin esperar respuesta.



Beneficios

- Anonimidad
- Concurrencia
- Escalabilidad de mensajes
- Soporte de procesos BATCH
- Soporta un BAJO ACOPLAMIENTO entre mensajes que producen el PRODUCTOR - CONSUMIDOR

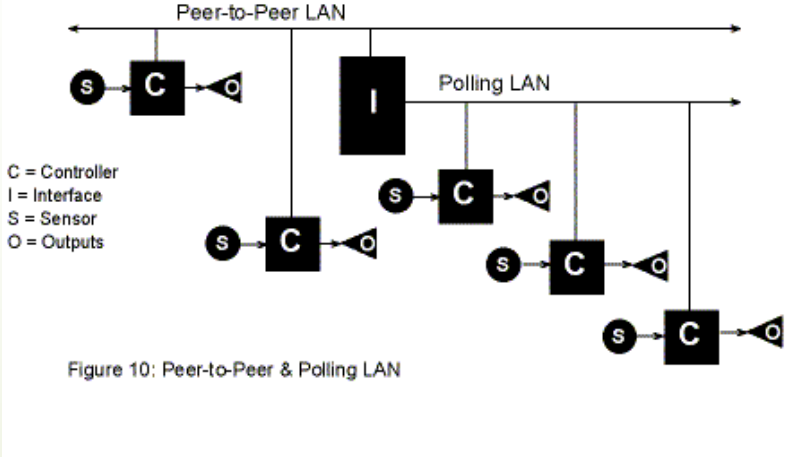
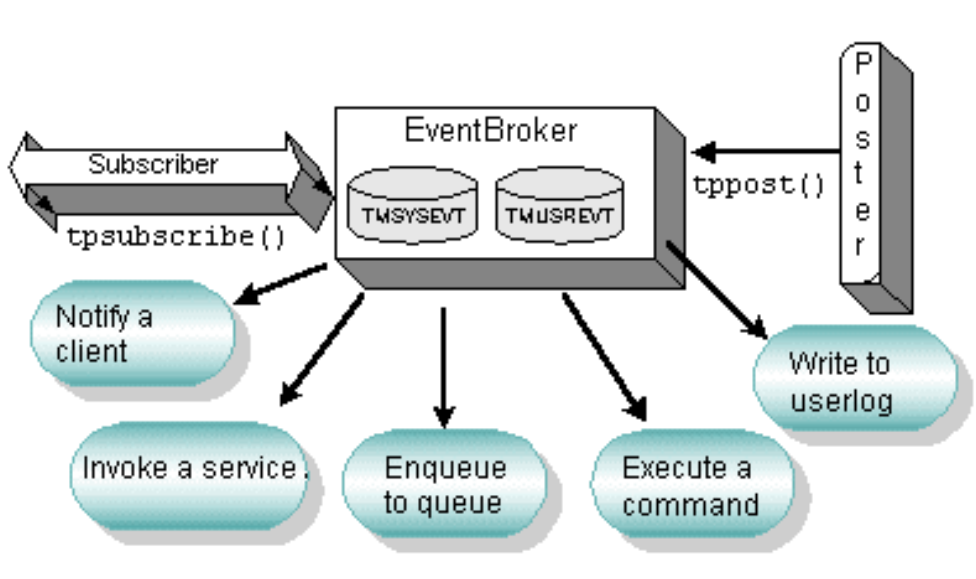


Figure 10: Peer-to-Peer & Polling LAN

Limitaciones

- Capacidad limitada de la COLA DE MENSAJES
- Separación completa de la presentación y la abstracción
- Incremento de la complejidad en el diseño e implementación del sistema



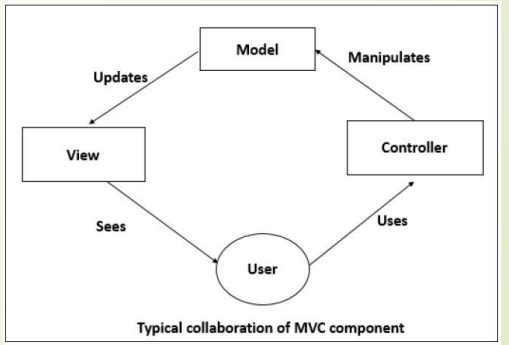


Arquitectura de software orientada a la interacción

M.Sc. Percy Huertas Niquén

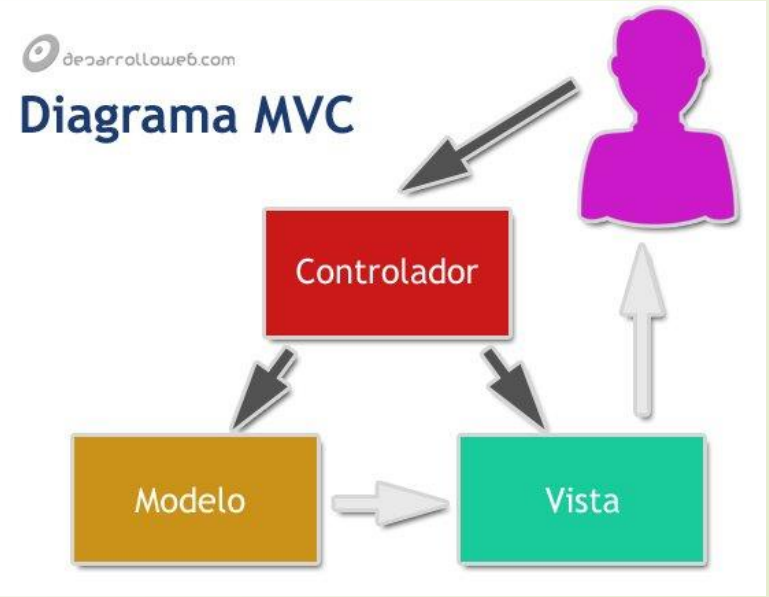
Introducción

- Muchos sistemas requieren la intervención del usuario en tiempo de ejecución
- Esta arquitectura descompone el sistema en tres particiones:
  - Módulo de datos
  - Módulo de control
  - Módulo de presentación
- Cada módulo presenta sus propias responsabilidades



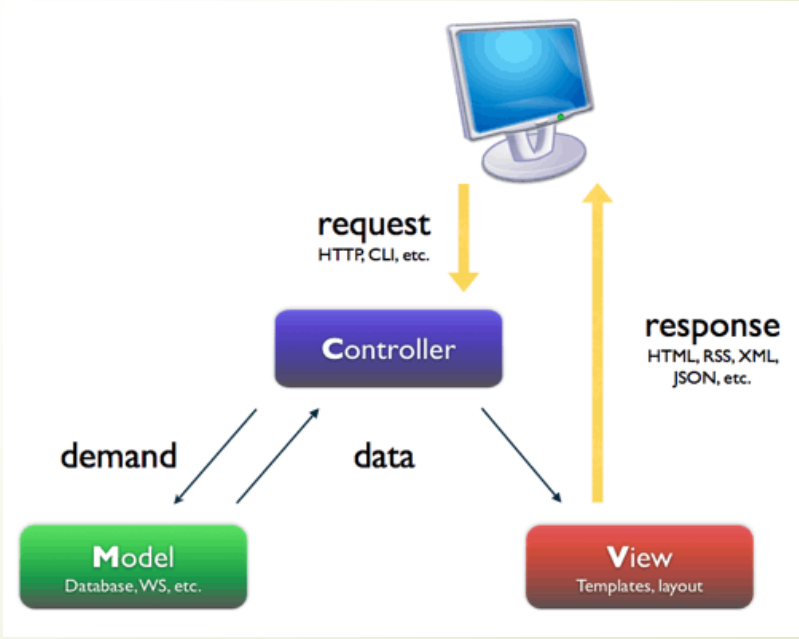
Modelo – vista - controlador

- Aplicado en sistemas donde los interfaces requieren de cambios en los datos
- Ejemplo Java Swing Components y Java Swing Layout



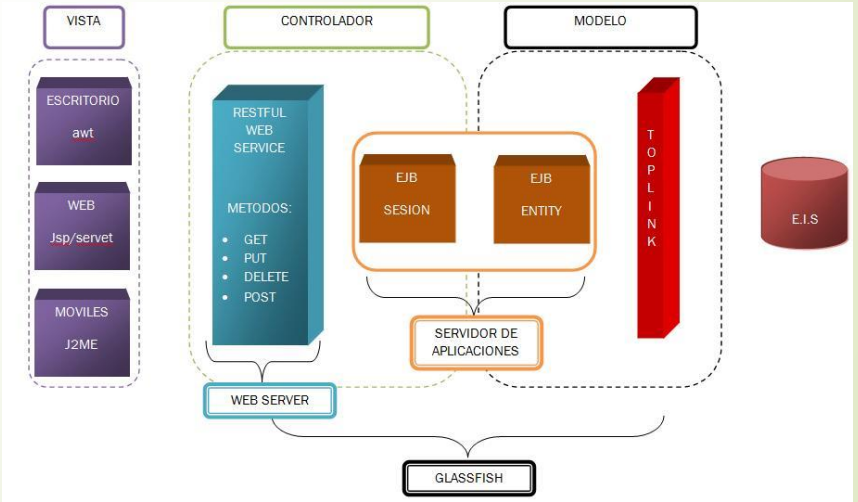
Aplicaciones

- Aplicaciones interactivas con múltiples vistas y cambios de datos en sus solicitudes
- Aplicaciones con una clara división entre controladores



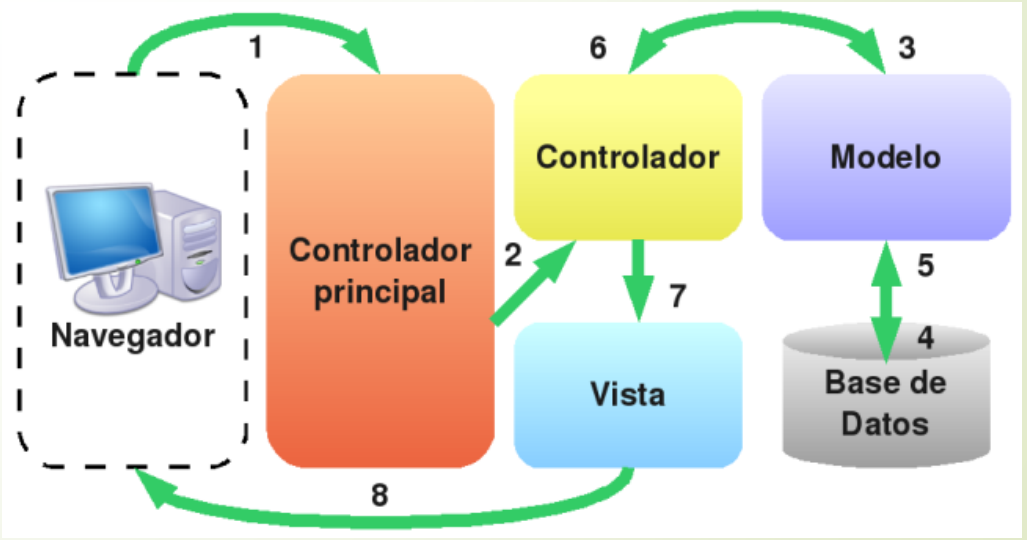
Beneficios

- Existen muchos vendedores en el mercado
- Muchas vistas son sincronizadas por el mismo modelo de datos
- Fácil de cambiar la información de las vistas
- Muy efectivo en desarrollo de software si existen equipos de trabajo dedicado a tareas específicas: programación, desarrollo y base de datos



Limitaciones

- No aplicable en sistemas orientados a AGENTES
- Muchos controladores y múltiples vistas causan inestabilidad en el sistema
- En algunos casos la división VISTA – CONTROLADOR no es clara



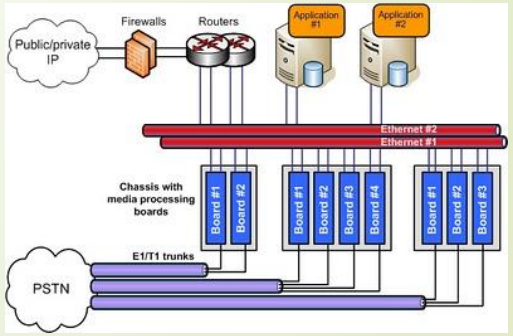


Arquitectura distribuida

M.Sc. Percy Huertas Niquén

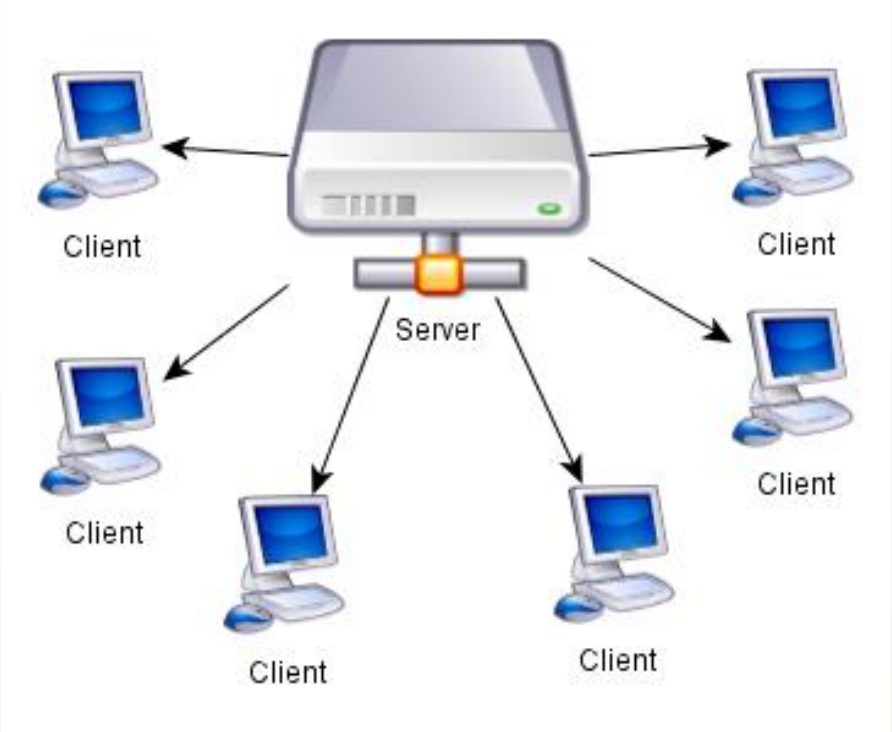
Introducción

- Un sistema distribuido es una colección de recursos computacionales y dispositivos de almacenamiento conectados por un sistema de comunicación en red
- Cada recurso puede emplear sus propios métodos de comunicación
- Los elementos que conforman un sistema distribuido son:
  - Red topológica
  - Modo de conexión
- En el mundo real muchos sistemas son distribuidos



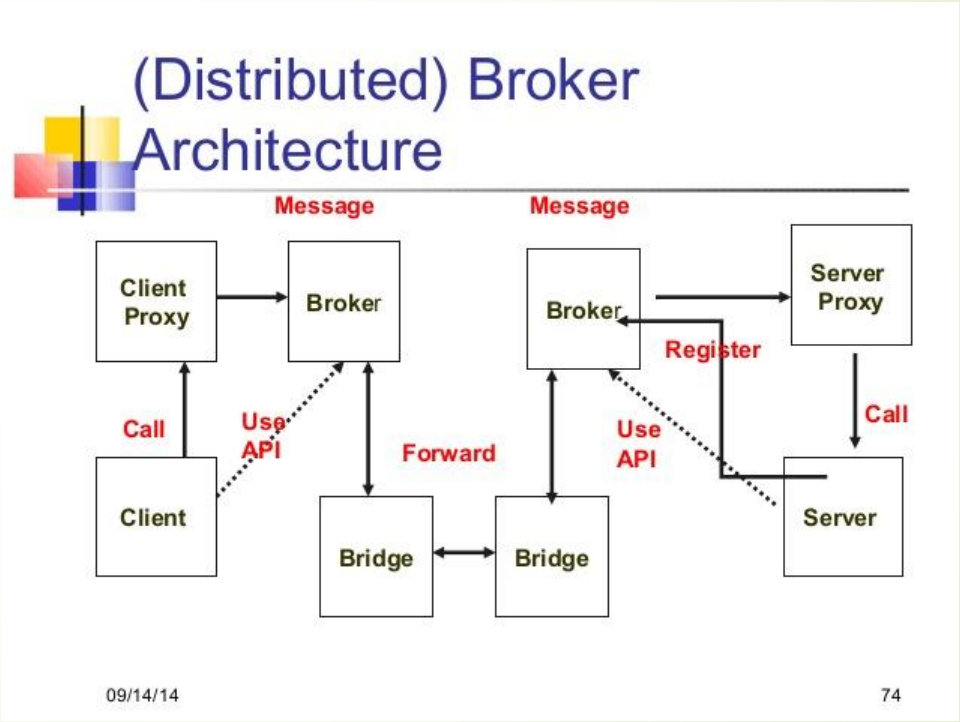
Cliente servidor

- Basado en dos procesos de comunicación: CLIENTE - SERVIDOR



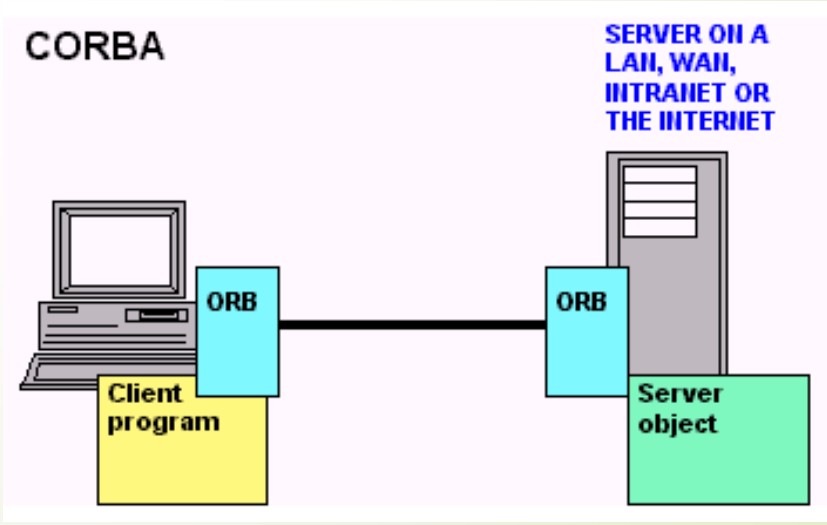
Estilo de la arquitectura corredor (Broker)

- Permite coordinar y facilitar la comunicación entre el CLIENTE - SERVIDOR



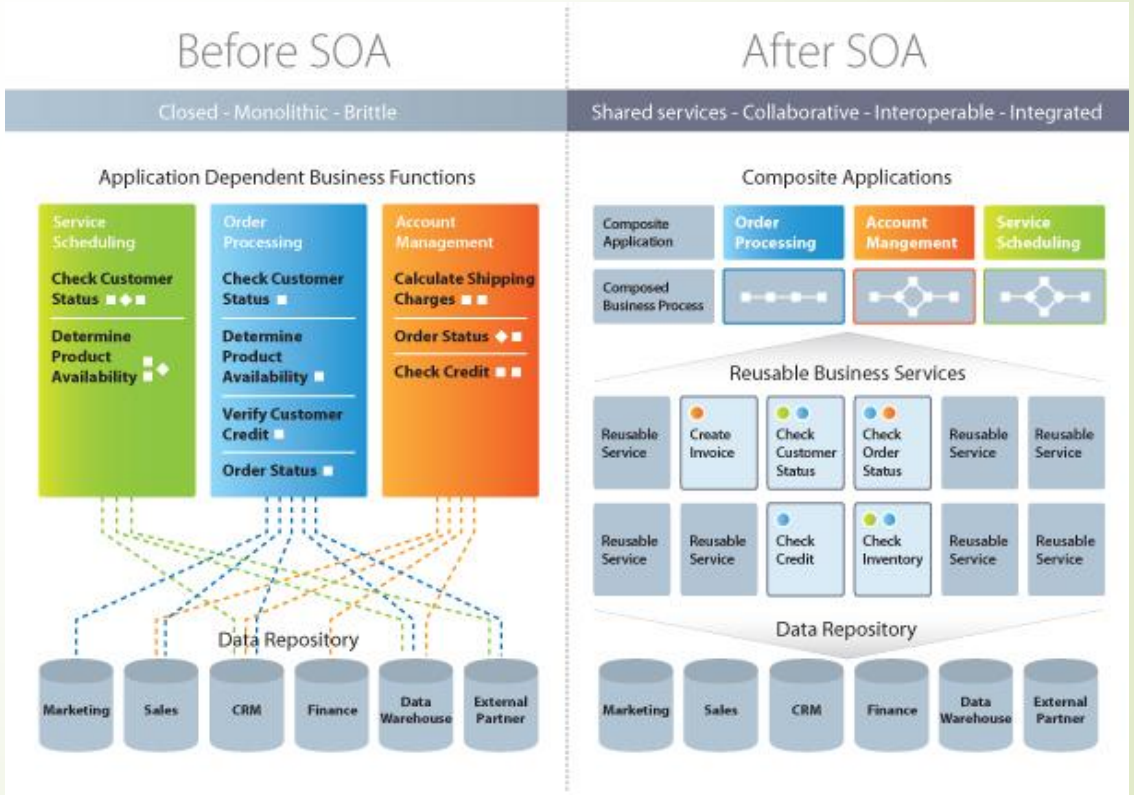
Broker en CORBA

- Es un estándar para la invocación de métodos remotos
- Presentan lenguajes independientes para la implementación de las aplicaciones
- Los interfaces se especifican en un lenguaje de definición de interfaces (IDL)



Arquitectura orientada a servicios (SOA)

- Se enfoca en los procesos de negocio



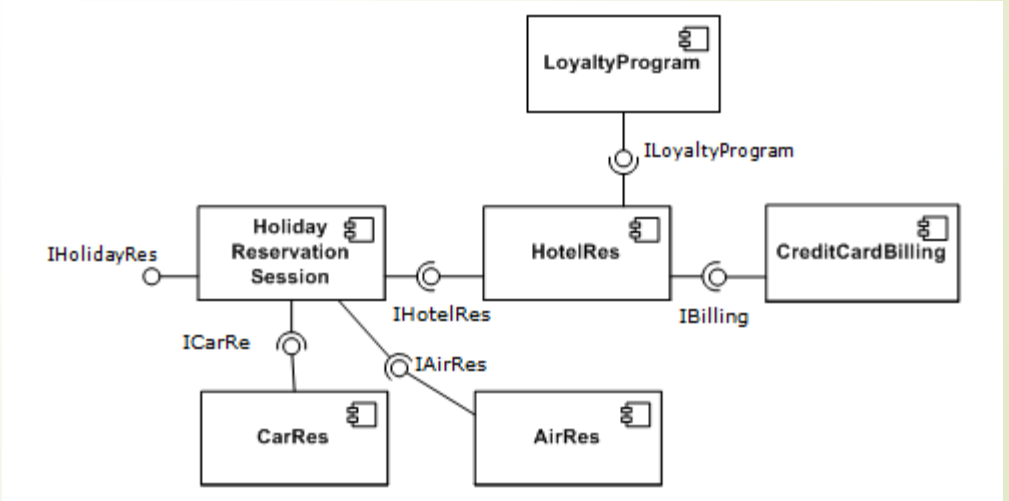


Arquitectura de software orientada a componentes

M.Sc. Percy Huertas Niquén

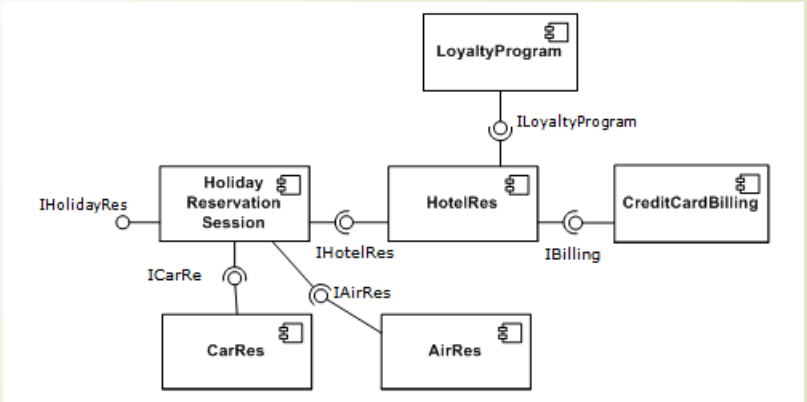
Introducción

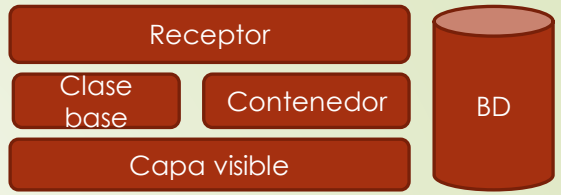
- Divide el problema en subproblemas, cada uno de ellos asociada con una previa partición de componentes
- El interfaz de componentes juega un rol importante
- El mejor motivo es la reutilización de componentes
- Incrementa la confiabilidad de los componentes si estos provienes de empresas serias



Principios del diseño orientado a componentes

- Componentes unidos a una arquitectura
- Partes de la interfaz de un componente para soportar la noción de arquitectura:
  - Tiempo de Composición
    - Elementos para generar una aplicación a partir de COTS
  - Tiempo de Diseño
    - Interfaces funcionales y dependencias de componentes
  - Tiempo de Ejecución
    - Servicios de composición dinámica en runtime





Parte 5: Arquitecturas heterogéneas y GUI's

M.Sc. Percy Huertas Niquén

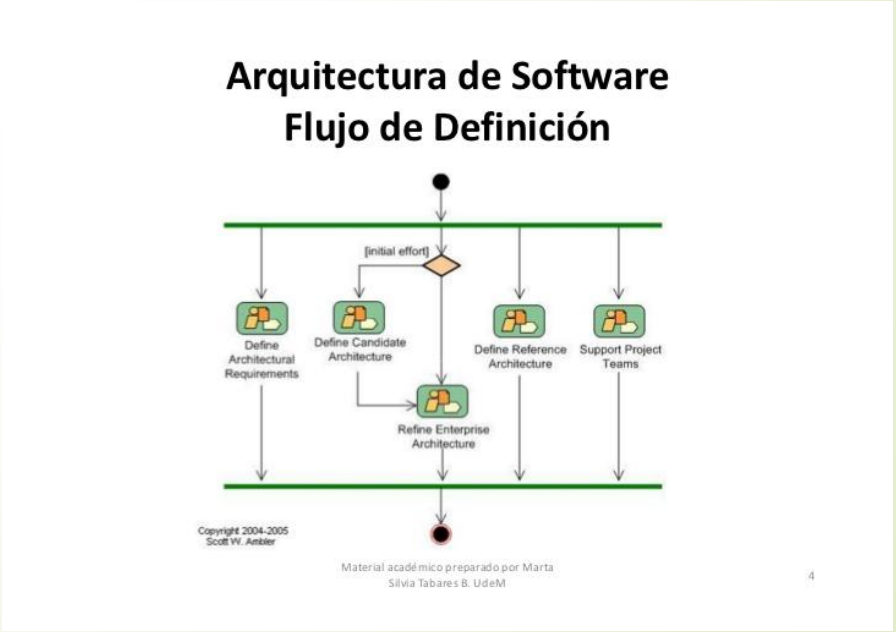


Arquitecturas heterogéneas

M.Sc. Percy Huertas Niquén

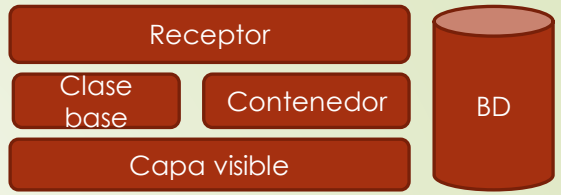
Introducción

- En un solo proyecto se pueden emplear diferentes estilos arquitectónicos
- En proyectos de larga escala es muy útil emplear este tipo de arquitecturas



Estilos heterogéneos

- Se incluye en este grupo formas compuestas o indóciles a la clasificación en las categorías habituales. Tipos:
  - Sistemas de Control de Procesos: . El objetivo de un sistema de esta clase es mantener ciertos valores dentro de ciertos rangos especificados, llamados puntos fijos o valores de calibración.
  - Arquitecturas Basadas en Atributos: Los estilos basados en atributos incluyen atributos de calidad específicos que declaran el comportamiento de los componentes en interacción.
  - Estilos Peer to Peer: Consiste por lo general en procesos independientes o entidades que se comunican a través de mensajes.
  - Arquitecturas Basadas en Recursos: Representational State Transfer o REST.



Arquitecturas de interfaces gráficas de usuario

M.Sc. Percy Huertas Niquén

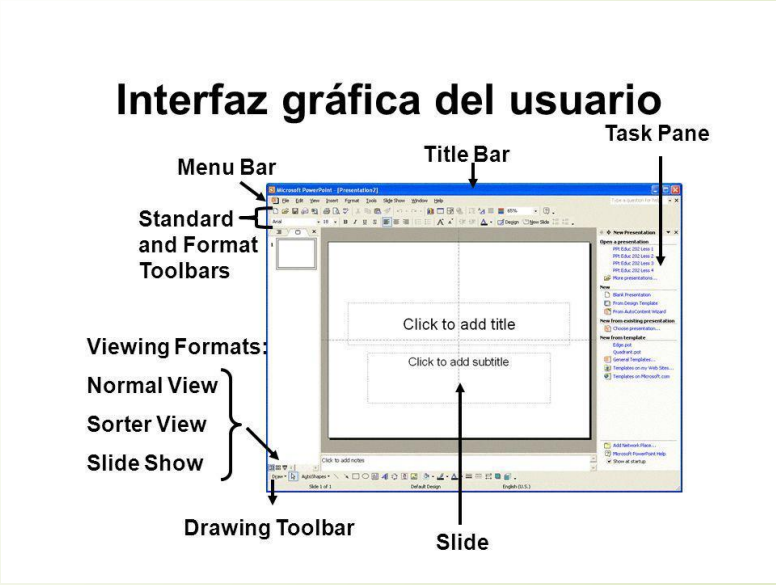
Introducción

- La impresión de los usuarios se da por los interfaces gráficos
- Objetivos de los interfaces de usuario:
  - Entrada de datos
  - Visualización de salidas de datos

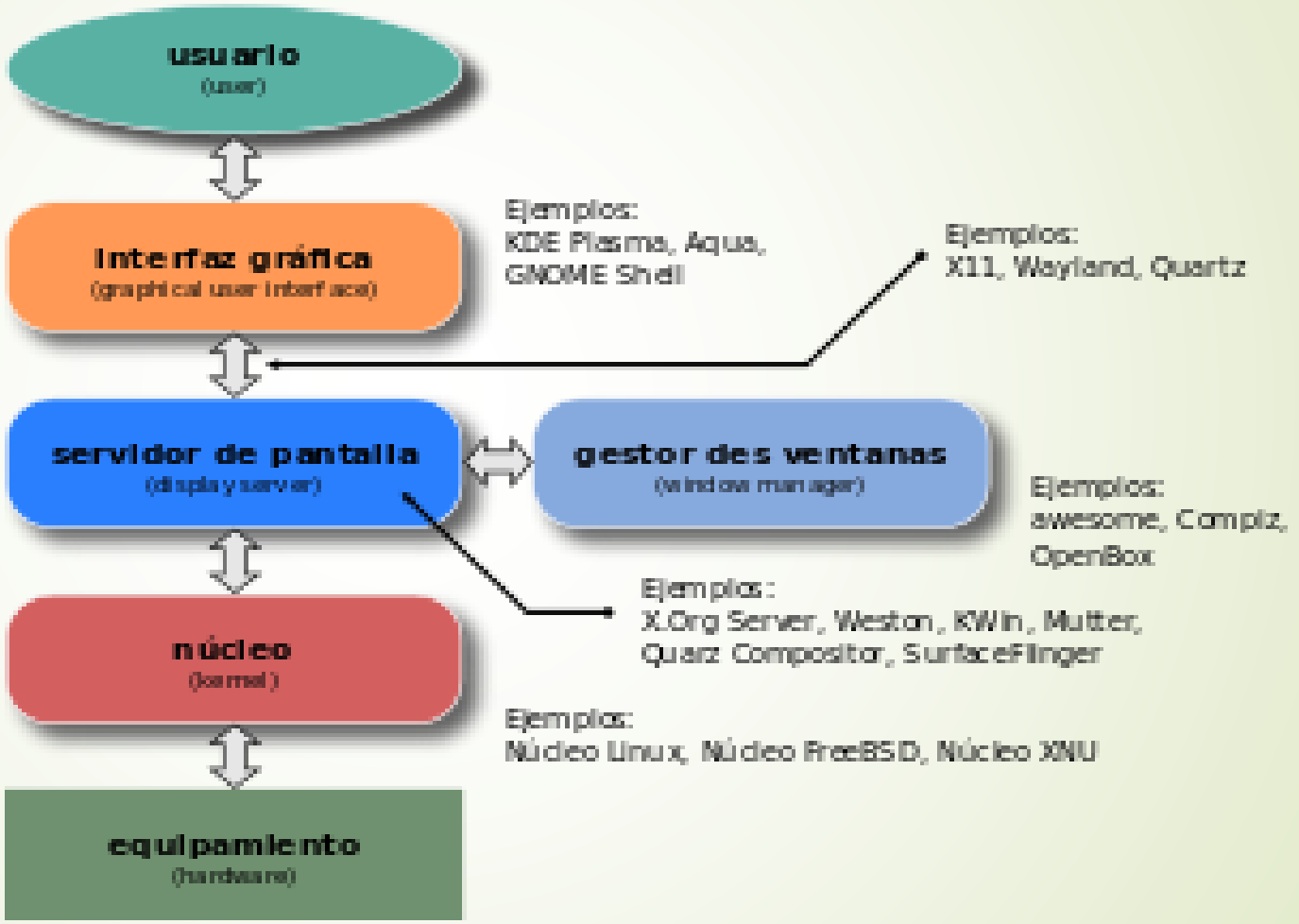


Interfaz gráfica de usuario

- La interfaz gráfica de usuario o GUI es un programa informático que actúa de interfaz de usuario, utilizando un conjunto de ventanas, imágenes y objetos gráficos para representar la información y acciones disponibles en la interfaz.
- Proporcionar un entorno visual sencillo para permitir la comunicación con el sistema operativo de una máquina o computador.



Ciclo de vida



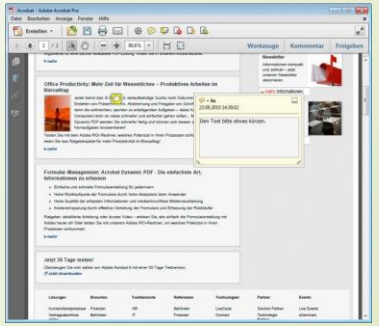
Interfaz grafica Web

- El diseño de una página Web tiene la esencial misión de vincular al usuario con el vasto mundo informativo, de un modo agradable y eficiente.
- Las herramientas a utilizar (imágenes estáticas y animadas, iconos, vínculos, etc.) por el usuario deben ser claras y fáciles de manejar.



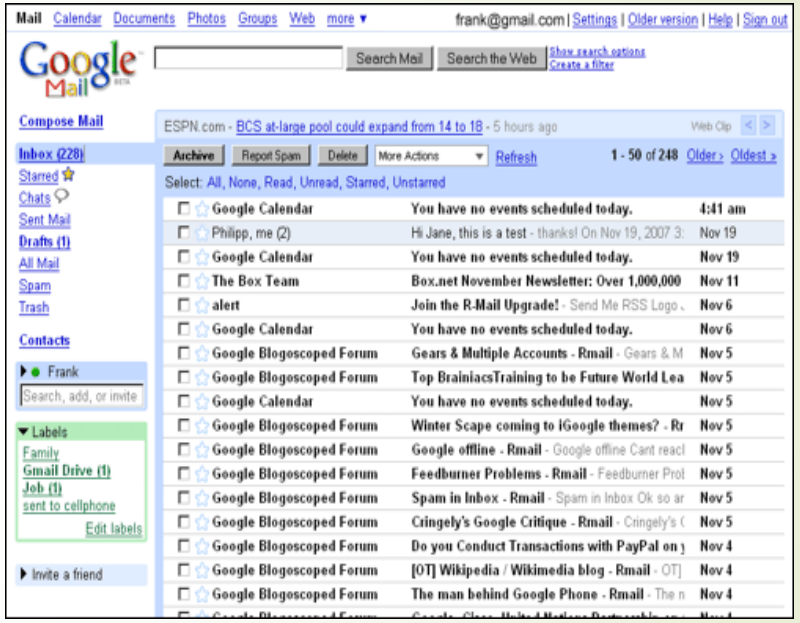
Para el correcto diseño de un sitio Web es necesario seguir una serie de etapas:

- El análisis de la información que tendrá el Web.
- La búsqueda y la estructuración de dicha información.
- El diseño informacional del sitio (diseño gráfico).
- La confección digitalizada de la interfaz.
- La publicación del sitio en Internet.
- La supervisión de la información del sitio (por el Webmaster)



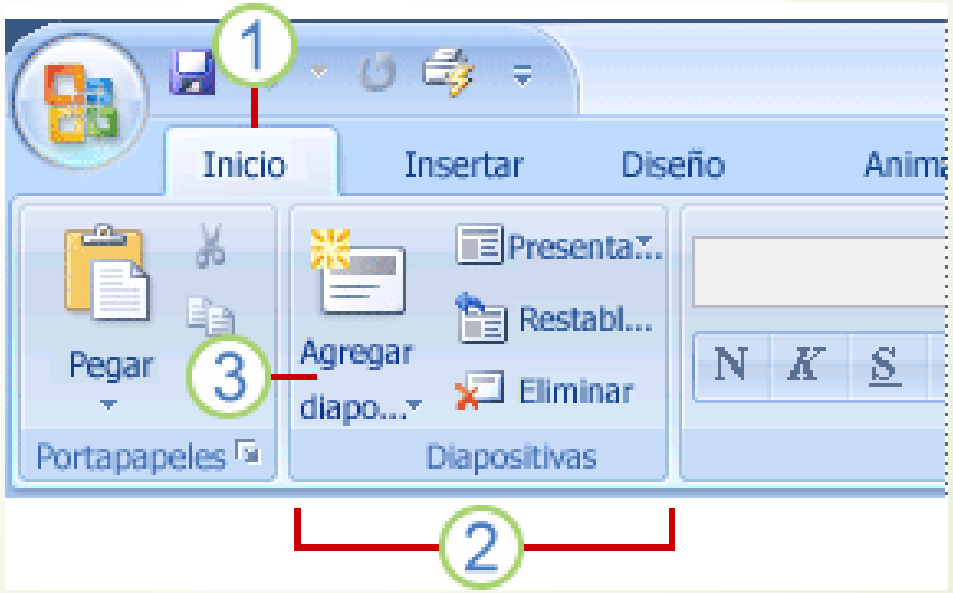
Criterios para una buena interfaz

- Control de usuario.
- Sensibilidad.
- Personalización.
- Dirección.
- Consistencia.
- Claridad.
- Retroalimentación.
- Indulgencia.
- Estética.
- Conciencia de las fortalezas y limitaciones humanas

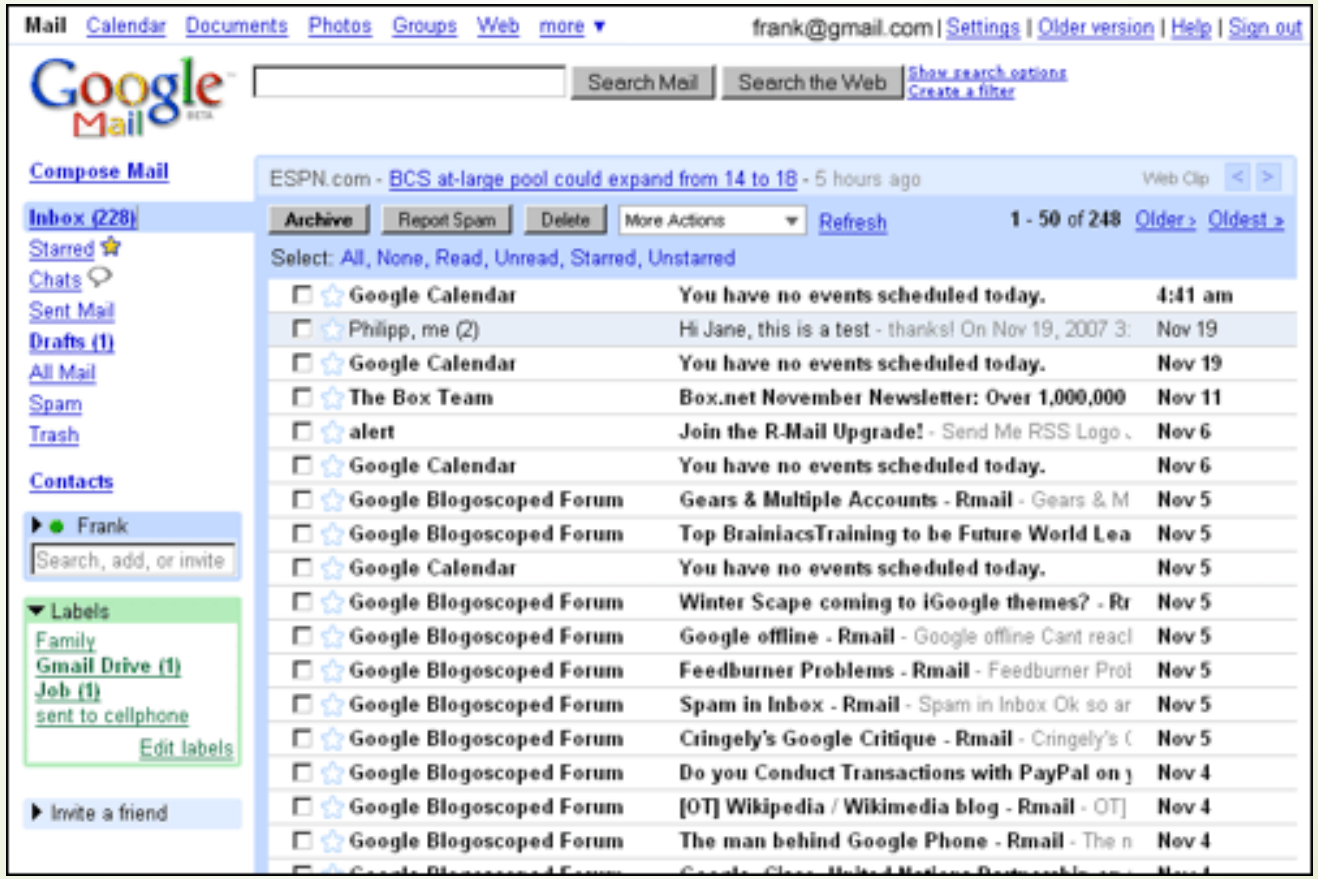


Además para una buena interfaz

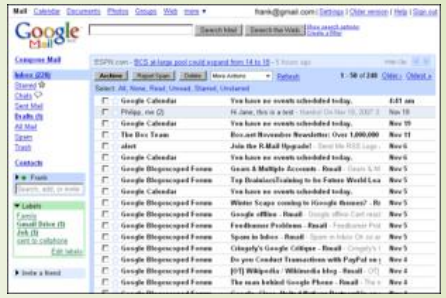
- opciones a elegir
- Panel de búsqueda, visible todo el tiempo
- Que sea intuitivo
- Buen diseño



Ejemplo: SOBRE GMAIL



Además de la importancia de...



- El diseño de formas
- Lograr una armonía agradable visualmente entre el fondo de las páginas y el contenido que se muestra en ella es un requisito fundamental.
- El diseño gráfico y tipográfico del Web es algo que también se descuida. El mal uso del color y el abuso de figuras, animados y fotos alteran el objetivo de atención del documento.

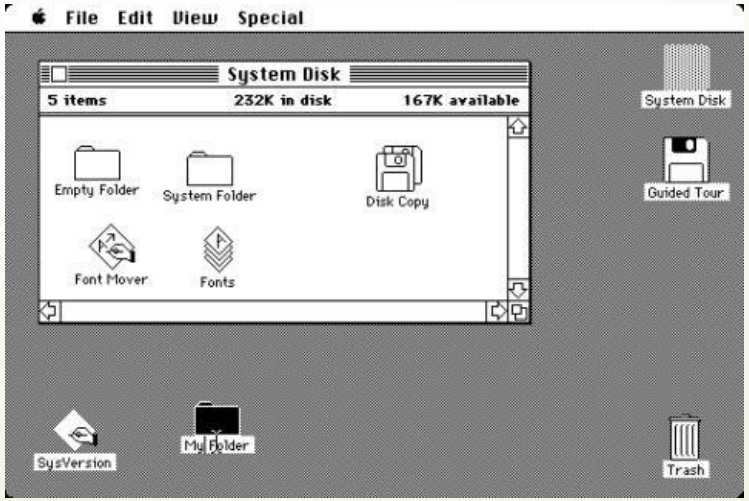
La tipografía

- Otro factor problemático son los títulos, subtítulos y encabezamientos de páginas que suelen resultar monótonos; se aconseja crear un énfasis ya sea cambiando el tamaño, el color, el estilo, o combinando tipos dentro de una palabra o frase, o en otro caso haciendo una combinación coherente de estos recursos.



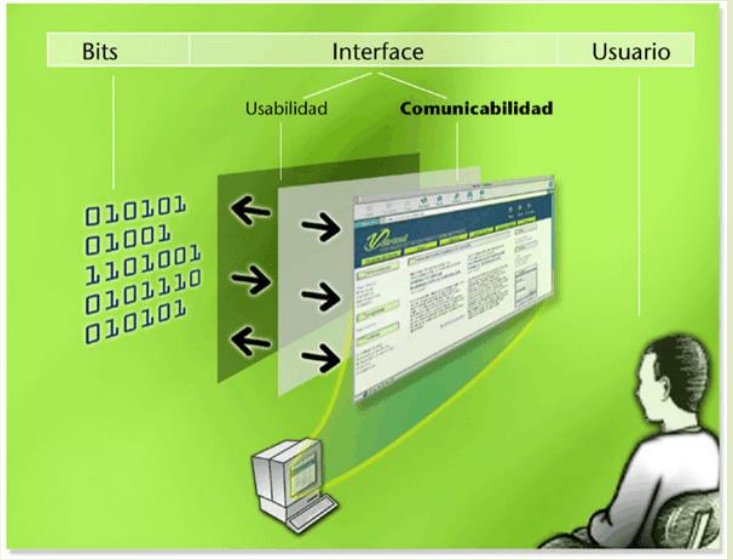
El diseño gráfico, el diseño sonoro y el diseño tridimensional

- Las ilustraciones gráficas han de tener una relación profunda con el contenido de la página.
- Es de cuidado que el sonido empleado sea coherente con el texto o la imagen con que esté relacionado
- Como soporte multimedia tiene esta rica posibilidad



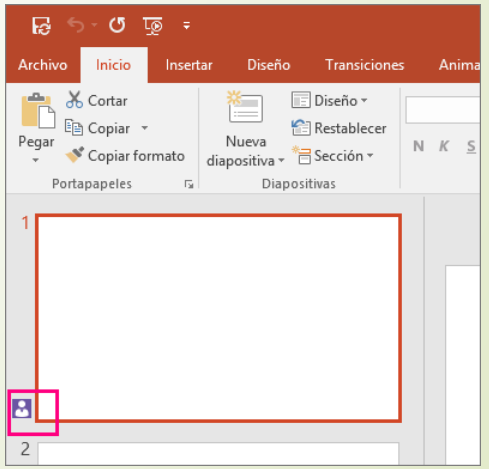
Los hechos que conducen a un mal diseño:

- No comprensión de lo que se quiere comunicar.
- La falta del valor semántico de los signos.
- La preferencia por lo estético.
- La subvaloración de lo funcional o viceversa.



Interfaz limpia:

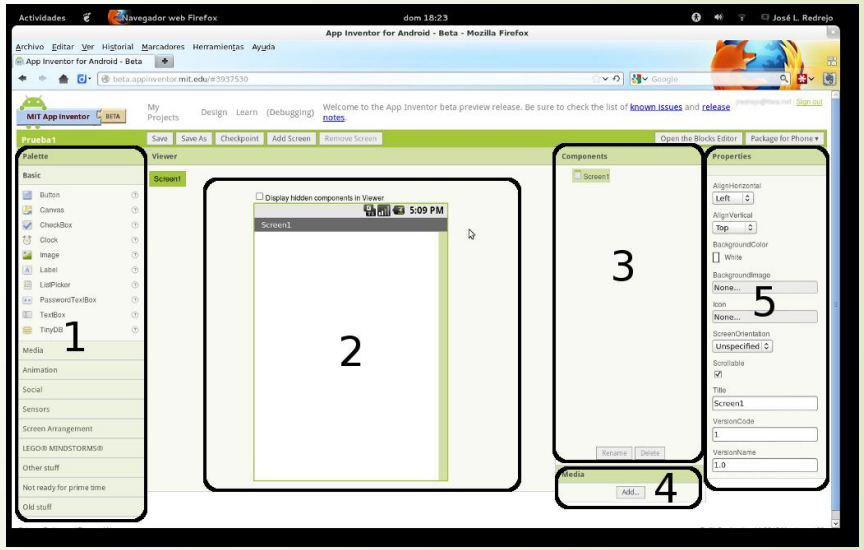
- La **interfaz es limpia, agradable y minimalista, con lo justo y necesario para trabajar**. Cabe destacar que, por mas que muchos servicios incorporen esta función en estos tiempos, **Gmail fue pionero en agrupar los mails como si fuesen conversaciones,** de manera que cada respuesta al e-mail original se ira solapando debajo de éste. Con esta practica se logra localizar los mensajes fácilmente y saber al mismo tiempo en relación con que tópico anterior nos están respondiendo.



Estilos de interfaces

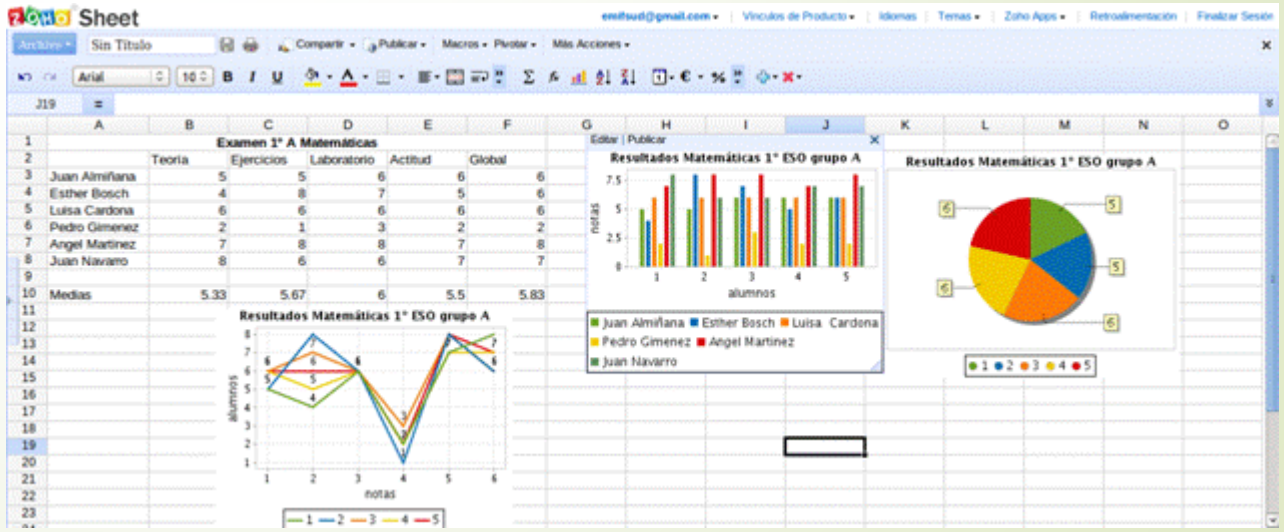
Los estilos de interfaces predominantes son:

- La interfaz por línea de comandos
- Menús y formularios
- Manipulación directa - *GUI*
- Interfaces con interacción asistida

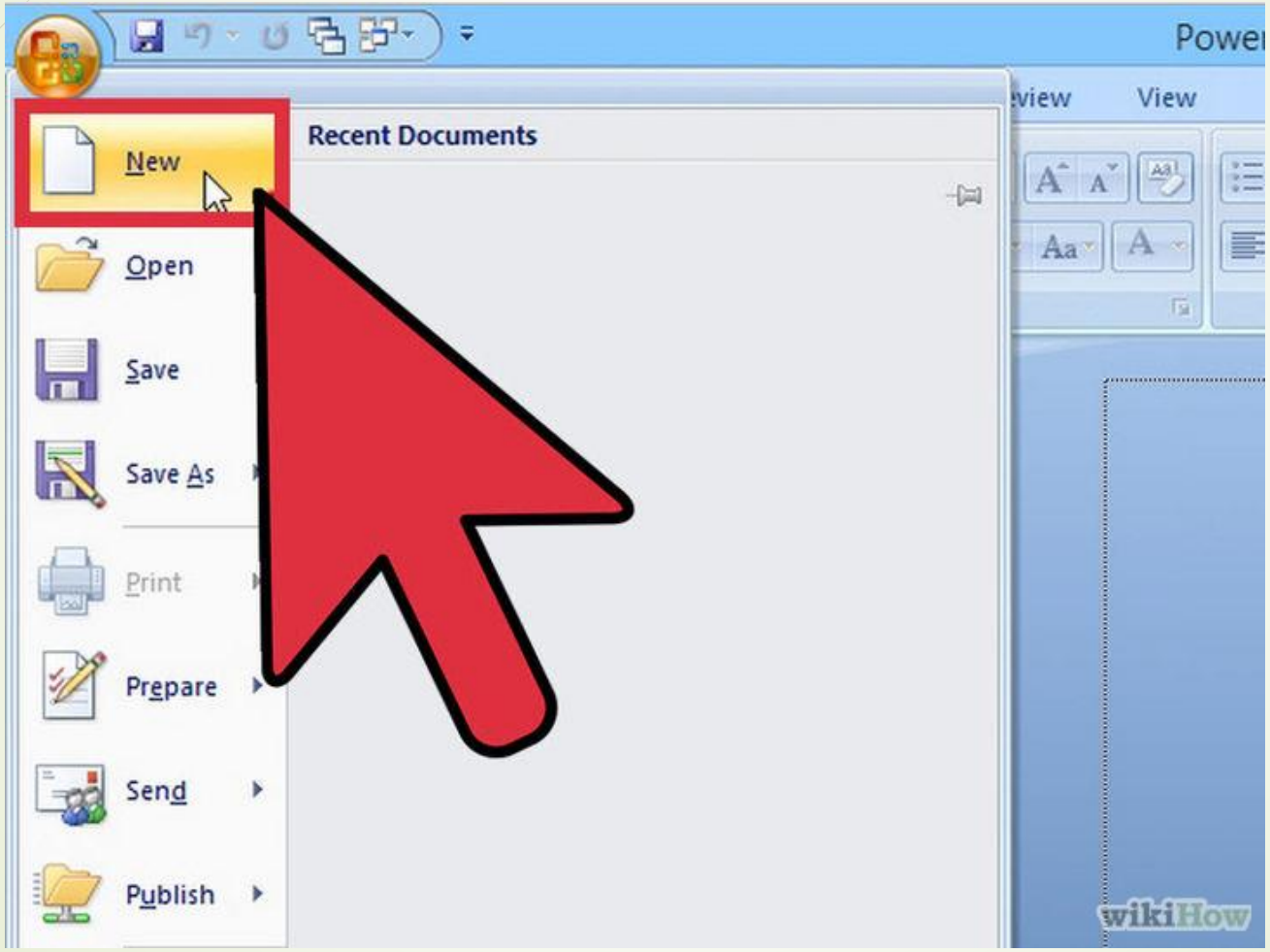


Objetivos de una buena interfaz:

- Maximizar la velocidad de aprendizaje
- Minimizar la tasa de errores
- Maximizar la velocidad de uso
- Estética adecuada

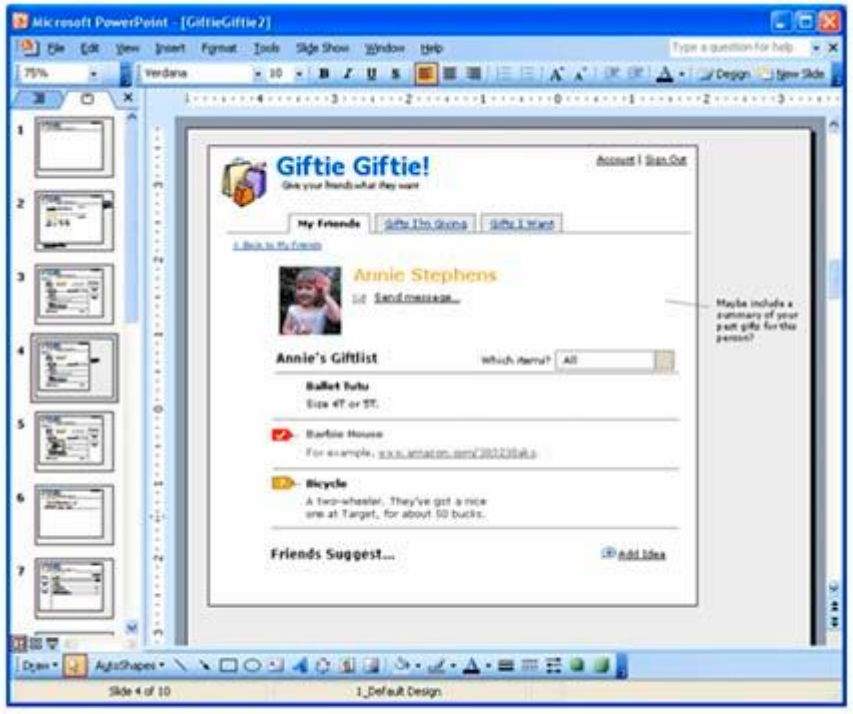


Diseño centrado en el usuario



Principios

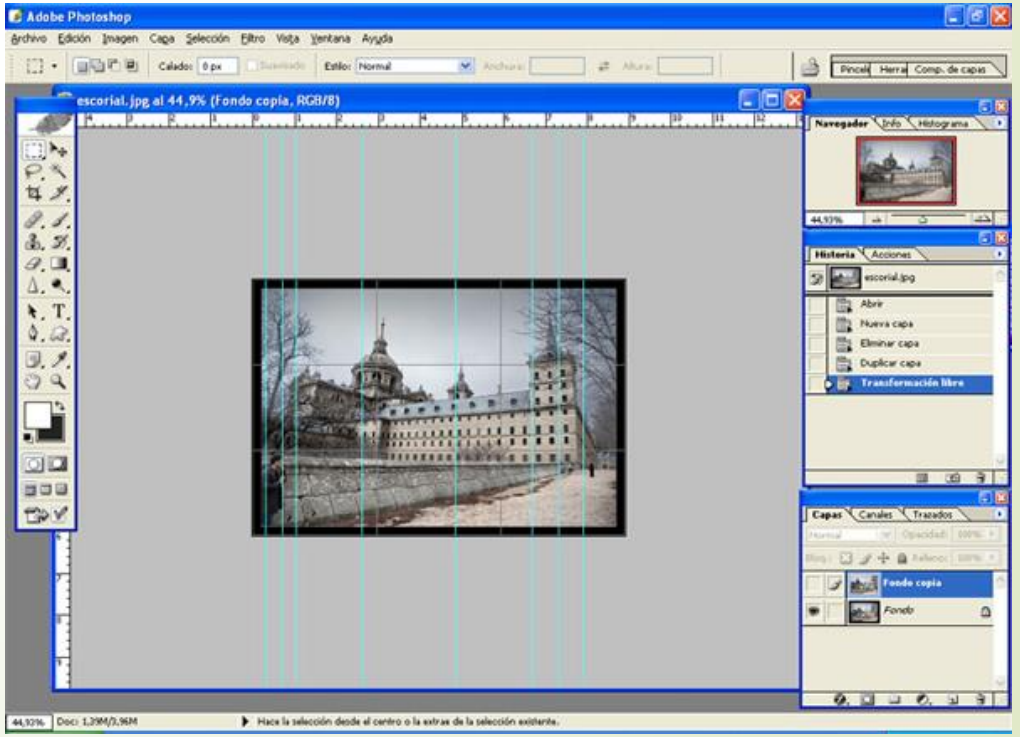
Son conceptos de muy alto nivel que deben ser utilizados en el diseño de aplicaciones.



Tomado de Boxes & Arrows

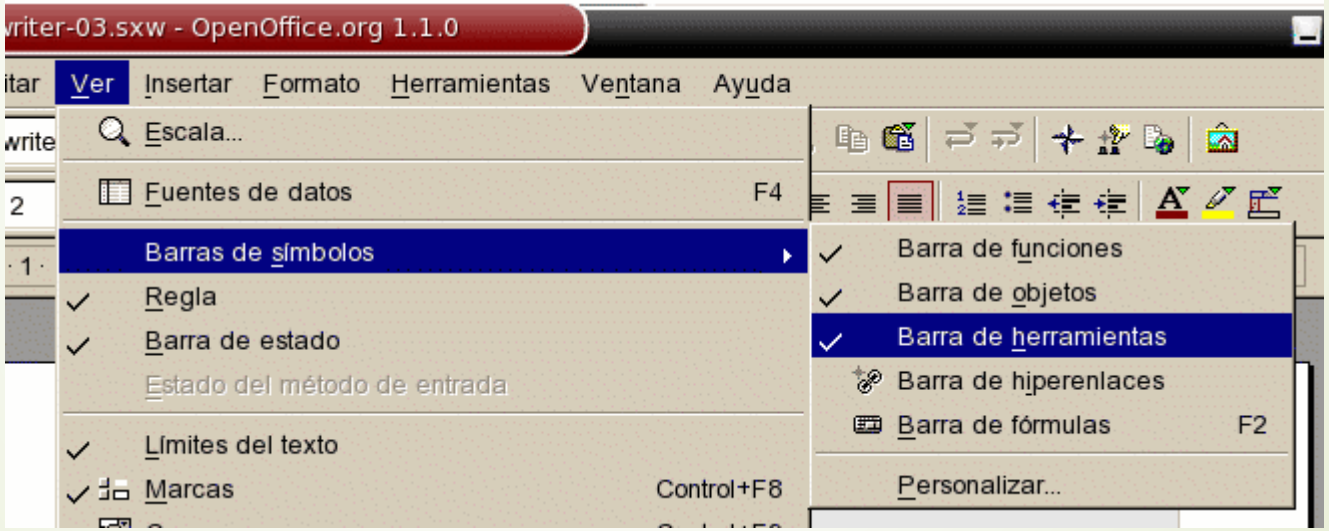
Reglas de diseño

Guían al diseñador con el fin de incrementar la "usabilidad". Se clasifican en estándares y directrices.



Estándares

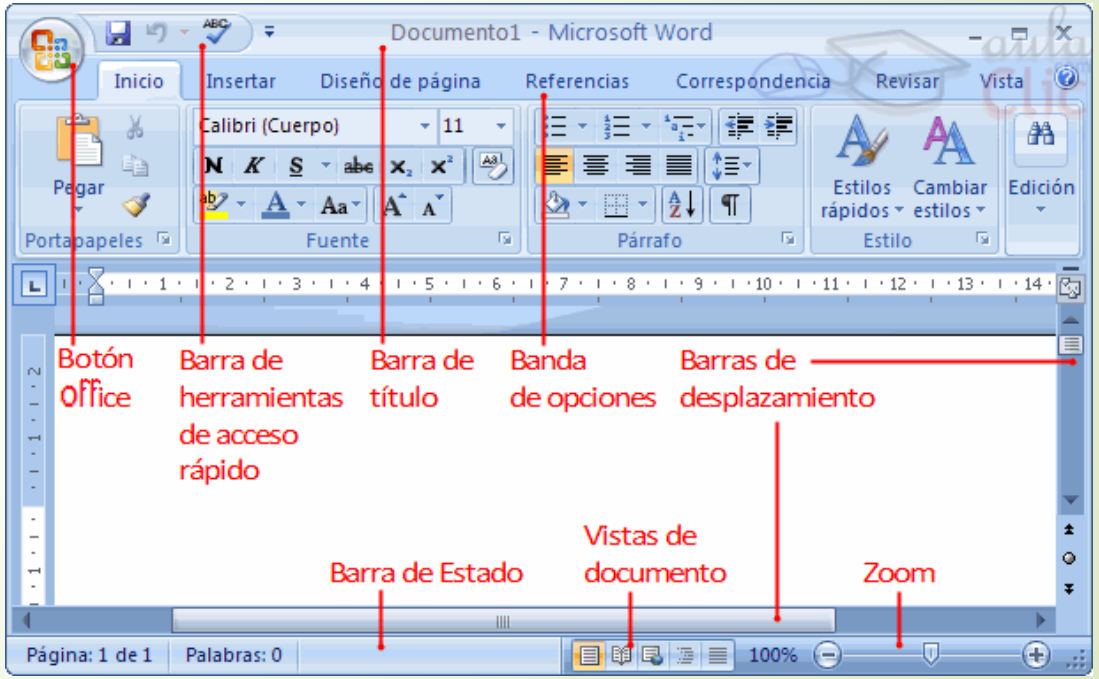
Son requisitos, reglas o recomendaciones basadas en principios probados y en práctica.



Directrices

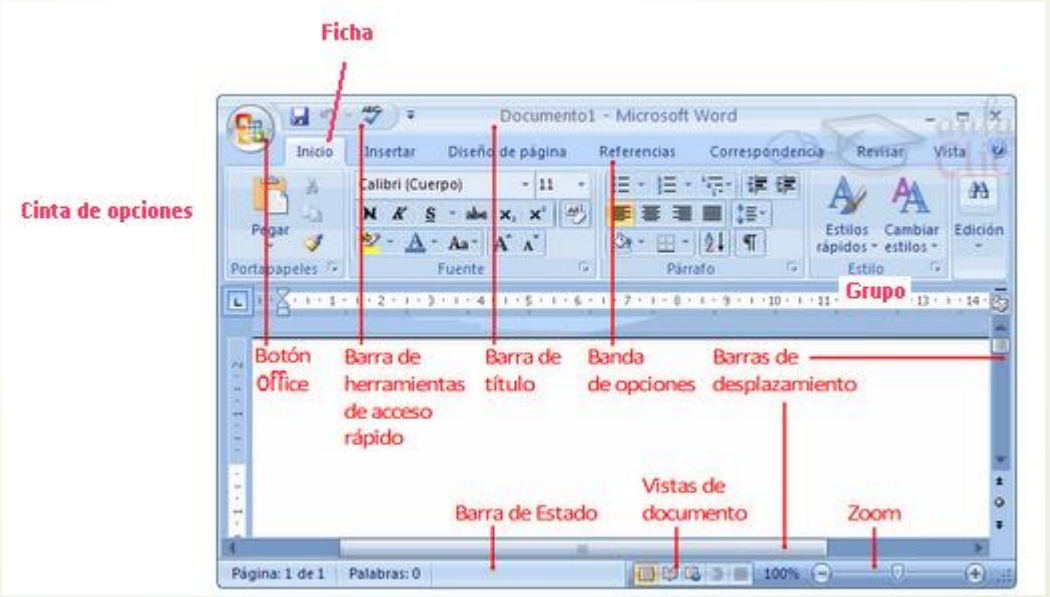
Las directrices recomiendan acciones que se basan en un conjunto de principios de diseño.

Son más específicas que los principios y requieren menos experiencia para entenderlas e interpretarlas que éstos.



Guías de estilo

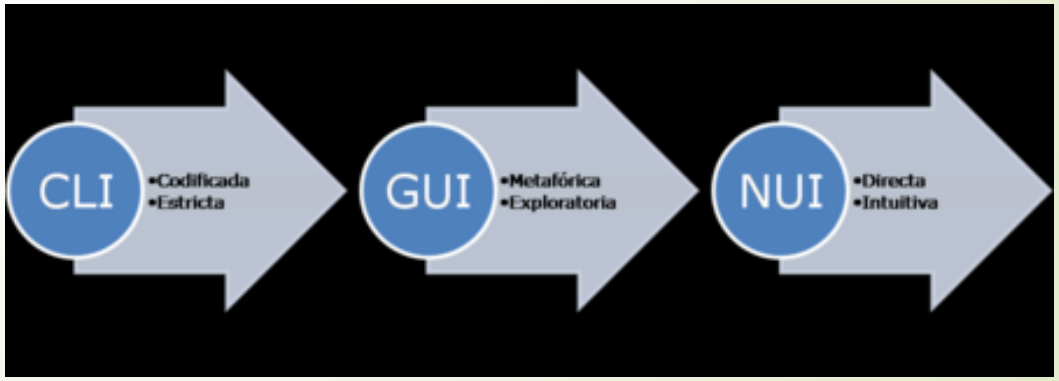
Llamadas también guías corporativas. Están basadas en principios y contienen directrices que se concretan a muy bajo nivel.



Principios de composición en el diseño

La composición se define como una distribución o disposición de todos los elementos que incluiremos en un diseño o composición de un menu o ventana, de una forma perfecta y equilibrada.

En un diseño, lo primero que se debe elegir son todos los elementos que aparecerán en él, luego debemos distribuirlos para colocarlos con el espacio disponible.
Los elementos pueden ser tanto imágenes, como espacios en blanco, etc. Es muy importante tener en cuenta de que forma situaremos estos elementos en nuestra composición, para que tengan un equilibrio formal y un peso igualado.





Parte 6: Arquitectura en la nube

M.Sc. Percy Huertas Niquén

Analogía

Ayer



Hoy



Características de la computación en la nube

- Demanda del auto servicio: Servicios unilaterales según demanda.
- Acceso ubicuo a la red: Acceso por mecanismos estándares.
- Recursos comunes: Servicio a múltiples consumidores.
- Independencia de lugares: Red puede estar en diferentes lugares.
- Rápida elasticidad: Agrupación de recursos.
- Servicio medido: Optimización de recursos.
- Multialquiler: Una aplicación para diferentes tipos de usuario.

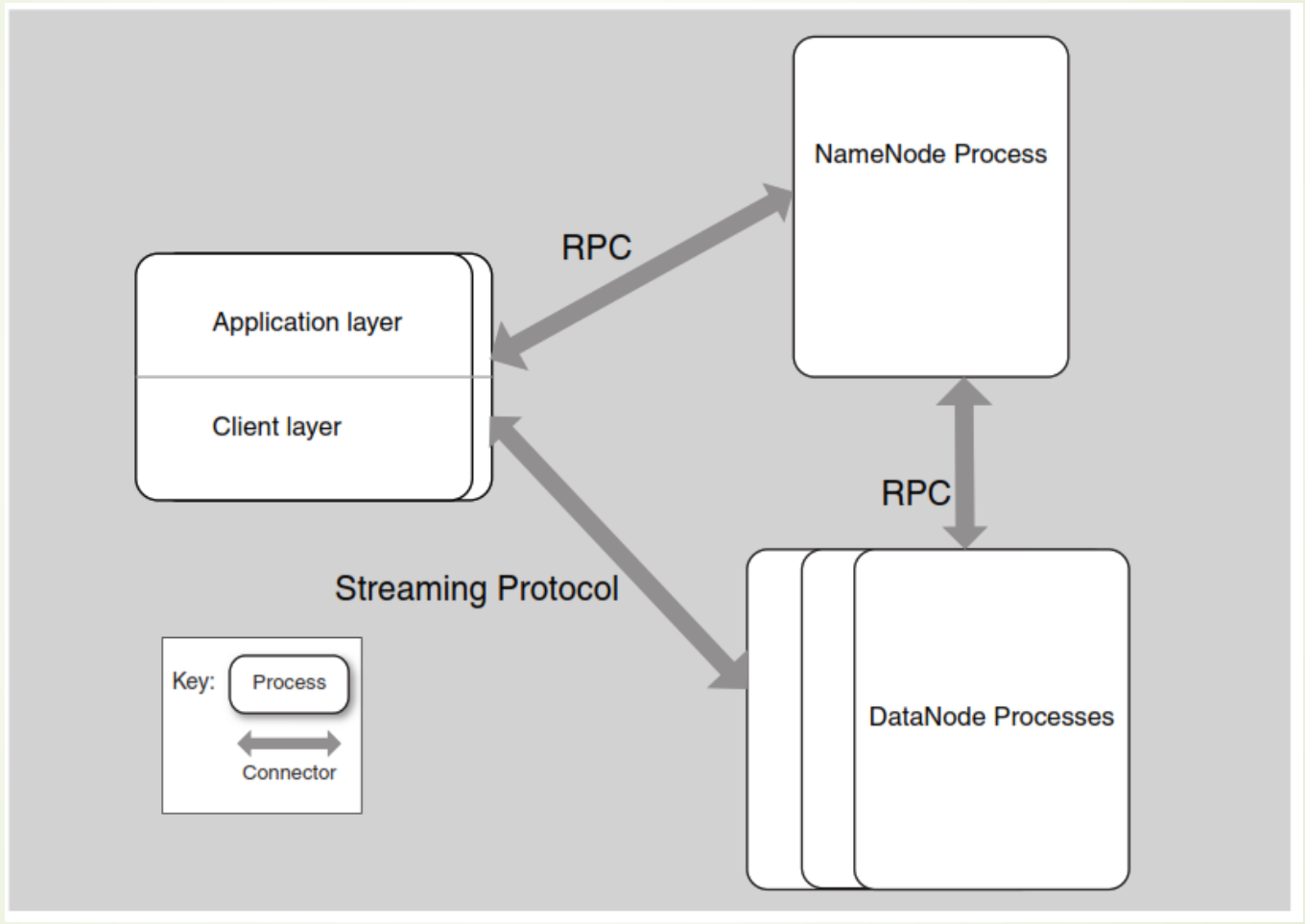
Modelos de servicios en la nube

- Software como servicio.
- Plataforma como servicio.
- Infraestructura como servicio.

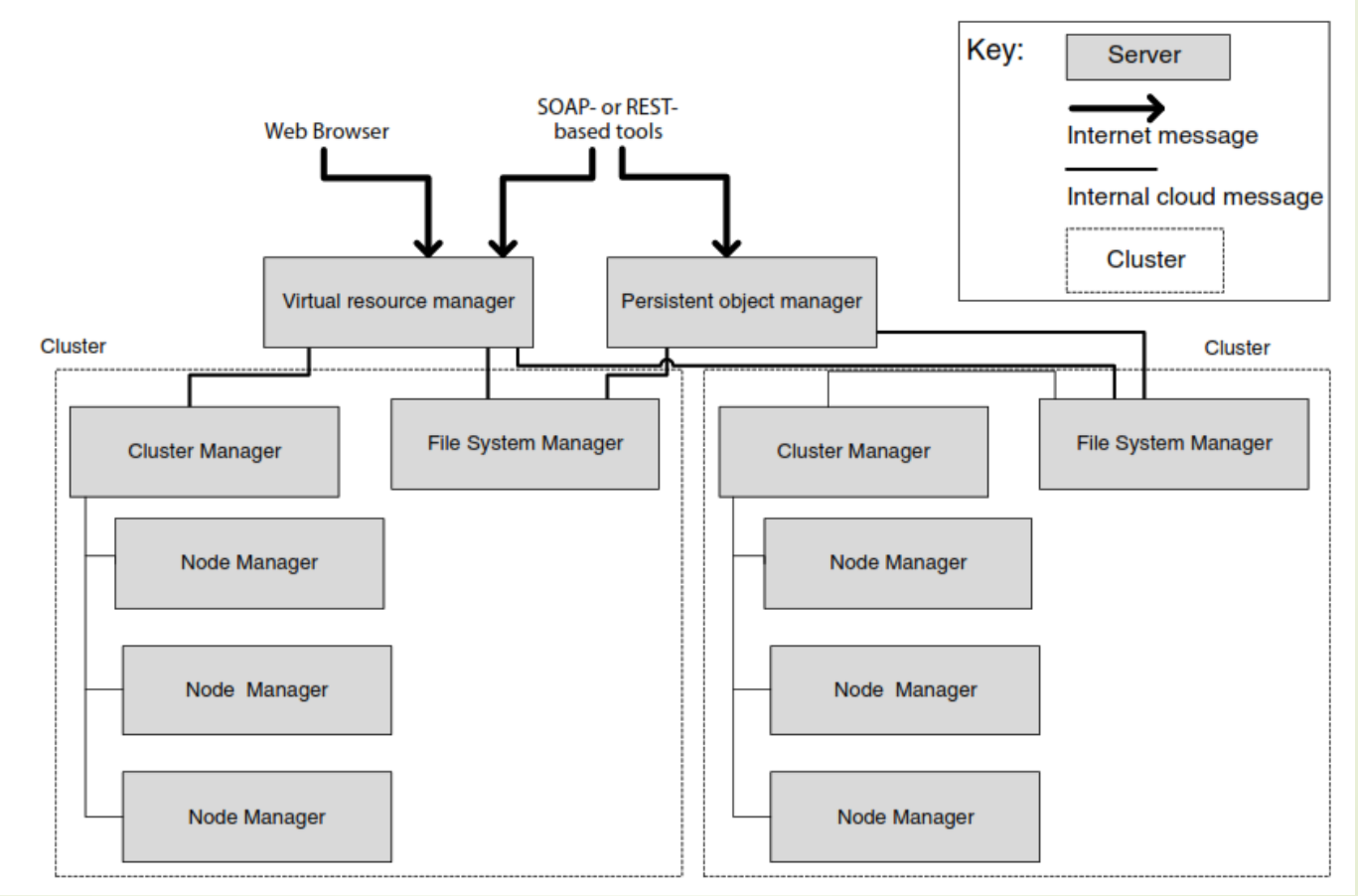
Modelos de implementación

- Nube privada
- Nube pública
- Nube comunitaria
- Nube híbrida

Vista de conectores y componentes



Ejemplo de Arquitectura: Infraestructura como servicio



Bibliografía

- Lixin, Tao (2010). Software and Architecture and Design Illuminated. 1/E. Jones and Bartell Illuminated Series. 2010.
- Sommerville, Ian (2011). Ingeniería de Software. Addison Wesley. Novena edición. España. 2011.
- http://www.enterate.unam.mx/Articulos/2006/febrero/arquitec.htm